

Update: June 6, 2026

PostgreSQL 19 New Features

With Examples (Beta 1)

Noriyoshi Shinoda

Index

Index.....	2
1. About This Document	5
1.1. Purpose	5
1.2. Audience	5
1.3. Scope	5
1.4. Software Version.....	5
1.5. The Question, Comment, and Responsibility	6
1.6. Notation	6
2. New Features Summary	8
2.1. Improvements to adapt to large scale environments	8
2.2. Improved reliability.....	8
2.3. Improved maintainability.....	9
2.4. Improvements in Programming	9
2.5. Incompatibility	10
2.5.1. Support Information.....	10
2.5.2. Transaction ID Round Warning.....	10
2.5.3. LSN Notation	11
2.5.4. LDAP protocol	11
2.5.5. Object Name	11
2.5.6. GUC.....	11
2.5.7. MD5 Authentication	12
2.5.8. Catalog pg_statistic_ext_data.....	12
2.5.9. ON CONFLICT Clause.....	13
2.5.10. RADIUS Authentication.....	14
2.5.11. Command psql.....	14
2.5.12. MULE_INTERNAL Encoding.....	15
2.5.13. COPY FROM Statement	15
2.5.14. JSON_ARRAY Function	16
3. New Feature Detail.....	17
3.1. Architecture.....	17
3.1.1. Changed System Catalogs	17
3.1.2. Logical Replication Enhancements.....	25
3.1.3. Partition Table Enhancements	30
3.1.4. SQL Property Graph	33

3.1.5. SSL/SNI	34
3.1.6. Dynamic Checksum Configuration.....	35
3.1.7. Roles	36
3.1.8. Libpq.....	37
3.1.9. I/O Processing	38
3.1.10. Automatic Vacuum.....	39
3.1.11. Hook.....	41
3.1.12. Faster processing	42
3.1.13. Log Output.....	43
3.1.14. OAuth Authentication	43
3.1.15. Wait Events	44
3.2. SQL Statement.....	45
3.2.1. ALTER DOMAIN statement.....	45
3.2.2. ALTER FOREIGN DATA WRAPPER statement.....	45
3.2.3. ALTER PUBLICATION statement.....	45
3.2.4. ALTER SUBSCRIPTION statement.....	46
3.2.5. ALTER TABLE statement	46
3.2.6. CHECKPOINT statement.....	48
3.2.7. COPY statement	49
3.2.8. CREATE PUBLICATION statement	52
3.2.9. CREATE SCHEMA statement	53
3.2.10. CREATE STATISTICS statement.....	54
3.2.11. DELETE / UPDATE statement	55
3.2.12. EXPLAIN statement	56
3.2.13. GRANT statement	60
3.2.14. INSERT statement	60
3.2.15. REPACK statement.....	61
3.2.16. SELECT statement.....	62
3.2.17. SET statement	64
3.2.18. VACUUM statement	64
3.2.19. WAIT FOR LSN statement	65
3.2.20. Functions	67
3.2.21. JSON	75
3.2.22. Data Types	77
3.2.23. PL/pgSQL	79
3.2.24. PL/Python	80

3.2.25. Optimizer	80
3.3. Configuration parameters.....	89
3.3.1. Added Parameters	89
3.3.2. Changed Parameters.....	91
3.3.3. Changed Default Values	92
3.3.4. Removed Parameters	93
3.4. Utilities	94
3.4.1. Oid2name	94
3.4.2. Pg_archivecleanup.....	94
3.4.3. Pg_createsubscriber	94
3.4.4. Pg_dump.....	96
3.4.5. Pg_dumpall	97
3.4.6. Pg_restore	97
3.4.7. Pg_upgrade	98
3.4.8. Pg_verifybackup.....	99
3.4.9. Pg_waldump	99
3.4.10. Psql	99
3.4.11. Vacuumdb	103
3.5. Contrib Modules	105
3.5.1. Auto_explain.....	105
3.5.2. Basic_archive.....	106
3.5.3. File_fdw	106
3.5.4. Injection_points.....	107
3.5.5. Pg_buffercache.....	108
3.5.6. Pg_plan_advice	109
3.5.7. Pg_stash_advice	112
3.5.8. Pg_stat_statements.....	115
3.5.9. Postgres_fdw.....	117
3.5.10. Miscellaneous.....	119
URL list	120
Change History	121

1. About This Document

1.1. Purpose

The purpose of this document is to provide information about the major new features of open-source RDBMS PostgreSQL 19 Beta 1 (19.0).

1.2. Audience

This document is written for engineers who already know PostgreSQL, such as installation, basic management, etc.

1.3. Scope

This document describes the major differences between PostgreSQL 18 (18.4) and PostgreSQL 19 Beta 1 (19.0). As a general rule, this document examines the features of behavior change. This document does not describe and verify all new features. In particular, the following new features are not included.

- Bugfix
- Performance improvement by changing internal behavior
- Improvement of regression test
- Operability improvement by the psql command tab input
- Improvement of the pgbench command
- Improvement of the documentation, modify typos in the source code
- Refactoring without a change in behavior

1.4. Software Version

The contents of this document have been verified for the following versions and platforms.

Table 1 Version

Software	Version
PostgreSQL	PostgreSQL 18.4 (for comparison) PostgreSQL 19 Beta 1 (19.0) (2026/06/01 20:05:06)
Operating System	Red Hat Enterprise Linux 9 Update 5 (x86-64)
Configure options	--with-ssl=openssl --with-lz4 --with-zstd --with-llvm --with-libxml --enable-injection-points --with-libnuma --with-liburing

1.5. The Question, Comment, and Responsibility

The contents of this document are not an official opinion of PostgreSQL Global Development Group. The author and affiliation company do not take any responsibility for the problem caused by the mistake of contents. If you have any comments for this document, please contact Noriyoshi Shinoda (noriyoshi.shinoda@gmail.com).

1.6. Notation

This document contains examples of the execution of the command or SQL statement. Execution examples are described according to the following rules:

Table 2 Examples of notation

Notation	Description
#	Shell prompt for Linux root user.
\$	Shell prompt for Linux general user.
Bold	The user input string.
postgres=#	The psql command prompt used by PostgreSQL users with the SUPERUSER attribute.
postgres=>	The psql command prompt used by PostgreSQL users without the SUPERUSER attribute.
<u>Underline</u>	Important output items.
<<PASSWORD>>	Replaced by password string.
...	Indicates that it is omitted.

The syntax is described in the following rules:

Table 3 Syntax rules

Notation	Description
<i>Italic</i>	Replaced by the name of the object which users use, or the other syntax.
[]	Indicate that it can be omitted.
{ A B }	Indicate that it is possible to select A or B.
...	General syntax. It is the same as the previous version.

2. New Features Summary

More than 200 new features have been added to PostgreSQL 19. This chapter provides an overview of typical new features and benefits. Details of the new features will be explained in "3. New Feature Detail".

2.1. Improvements to adapt to large scale environments

The following features have been added that can be applied to large scale environments:

- Splitting and Merging Partitions

The feature to merge multiple partitions and split a single partition has been implemented for a partitioned tables.

- Dynamic Table Configuration Changes

It is now possible to modify the evaluation method for virtual columns and the ENFORCED setting for CHECK constraints using the ALTER TABLE statement.

- Use of Parallel Workers in Automatic VACUUM

In previous versions, parallel workers were not used during automatic VACUUM. In PostgreSQL 19, automatic VACUUM now supports parallel processing.

2.2. Improved reliability

PostgreSQL 19 implements the following enhancements to improve reliability.

- Addition of Output Formats for the pg_dumpall Command

The pg_dumpall command now supports output formats other than text. If a format other than text is selected, the restore operation uses the pg_restore command.

- SSL/SNI Support

PostgreSQL server-side support for SSL/SNI has been added. The pg_hosts.conf file is now available for configuring SNI settings.

2.3. Improved maintainability

The following features that can improve operability have been added.

☐ Sequence Replication and Excepted Tables

Sequence replication is now supported in logical replication environments. it is now possible to specify tables that are excepted from replication on the publication side.

☐ REPACK Statement

The REPACK statement has been added to execute VACUUM FULL and CLUSTER statements simultaneously.

2.4. Improvements in Programming

The following features have been added to SQL statements.

☐ SQL Property Graph

PROPERTY GRAPH objects, which execute graph queries, can now be created. The GRAPH_TABLE function is provided for PROPERTY GRAPH queries.

☐ Enhancement to the INSERT Statement

The DO SELECT clause can now be specified in the INSERT ON CONFLICT statement.

☐ Enhancement to the UPDATE / DELETE Statements

The ON PORTION OF clause, which enables partial updates of the range, is now supported in UPDATE and DELETE statements.

☐ GROUP BY ALL Clause

The GROUP BY ALL clause, which executes GROUP BY processing on all output columns, is now supported.

☐ New Contrib Modules

The Contrib modules pg_plan_advice and pg_stash_advice, which control execution plans, have been added.

2.5. Incompatibility

In PostgreSQL 19, the following specifications have been changed from PostgreSQL 18.

2.5.1. Support Information

PostgreSQL 19 changes the supported versions for the following platforms and tools

The changes in component support versions required to build PostgreSQL 19 are as follows.

- Meson 0.57.2 or higher [[f039c22](#)]
- gcc 4.7 or higher [[4fbc015](#)]
- C11 standard [[f5e0186](#)]
- Visual Studio 2019 or higher [[8fd9bb1](#)]

The following components are now supported in PostgreSQL 19.

- Tcl 9 support added [[32a2aa7](#)]

PostgreSQL 19 has restored support for the following operating systems.

- IBM AIX [[4a1b05c](#)]

PostgreSQL 19 now supports the following standards.

- UNICODE 17.0.0 [[57ee397](#)]
- GB18030 2022 [[5334620](#)]
- C23 standards [[76f4b92](#)]

2.5.2. Transaction ID Round Warning

The DETAIL log now includes a percentage remaining in the transaction ID round warning message. Additionally, the threshold for triggering the warning has been changed from 40 million to 100 million remaining IDs. [[e646450](#), [48f11bf](#)]

Example 1 Transaction Round Warning Log Message

```
WARNING: database "postgres" must be vacuumed within 3001666 transactions
DETAIL: Approximately 0.14% of transaction IDs are available for use.
HINT: To avoid transaction ID assignment failures, execute a database-wide
VACUUM in that database.
You might also need to commit or roll back old prepared transactions, or drop
stale replication slots.
```

2.5.3. LSN Notation

The notation for LSN has been standardized, and the lower 32 bits are now always zero-padded when displayed. This change breaks compatibility with the notation used in some views and function return values. [[2633dae](#)]

Example 2 LSN Notation

```
postgres=# SELECT * FROM
           pg_create_logical_replication_slot('slot1', 'pgoutput') ;
 slot_name |      lsn
-----+-----
 slot1     | 0/019D2F30
(1 row)
```

2.5.4. LDAP protocol

The connection to the LDAP server now uses LDAP version 3. Previously, version 2 was used. [[5f19d13](#)]

2.5.5. Object Name

The CR and LF characters can no longer be used in the names of databases, roles, and tablespaces. [[b380a56](#)]

2.5.6. GUC

PostgreSQL 19 introduces the following incompatibilities related to parameter (GUC) settings.

- Parameter `standard_conforming_strings`

This parameter can no longer be set to "off". [[4576208](#)]

Example 3 Changes to the `standard_conforming_strings` parameter

```
postgres=> SET standard_conforming_strings = off ;  
ERROR: non-standard string literals are not supported
```

- Resetting Parameters

Specifying a parameter that does not exist in the `ALTER DATABASE RESET` or `ALTER USER RESET` statements will result in an error. In previous versions, this did not result in an error. [[9a71989](#)]

Example 4 Execute `ALTER DATABASE/USER RESET` statement

```
postgres=# ALTER DATABASE postgres RESET foo ;  
ERROR: unrecognized configuration parameter "foo"  
postgres=# ALTER USER postgres RESET foo ;  
ERROR: unrecognized configuration parameter "foo"
```

2.5.7. MD5 Authentication

When the `md5_password_warnings` parameter is set to "on" (the default value), the warning is now output during authentication using an MD5 password. [[bc60ee8](#)]

Example 5 Warning when using MD5 authentication

```
$ psql -d postgres -U user1 -h dbsvr1  
Password for user user1: <<PASSWORD>>  
WARNING: authenticated with an MD5-encrypted password  
DETAIL: MD5 password support is deprecated and will be removed in a future  
release of PostgreSQL.  
psql (19beta1)  
Type "help" for help.
```

2.5.8. Catalog `pg_statistic_ext_data`

The JSON output format for the `stxdndistinct` and the `stxddependencies` columns in the `pg_statistic_ext_data` catalog has been changed. [[1f927cc](#), [e76defb](#)]

Example 6 PostgreSQL 18 Output Formats

```
postgres=# SELECT jsonb_pretty(d.stxdndistinct::text::jsonb) AS stxdndistinct,
jsonb_pretty(d.stxddependencies::text::jsonb) AS stxddependencies
FROM pg_statistic_ext s LEFT JOIN pg_statistic_ext_data d ON (d.stxoid = s.oid)
WHERE s.stxname = 'data1_col1_col2' ;
 stxdndistinct | stxddependencies
-----+-----
{              +| {              +
  "1, 2": 10+|  "1 => 2": 1.000000+
}              | }
(1 row)
```

Example 7 PostgreSQL 19 Output Formats

```
postgres=# SELECT jsonb_pretty(d.stxdndistinct::text::jsonb) AS stxdndistinct,
jsonb_pretty(d.stxddependencies::text::jsonb) AS stxddependencies
FROM pg_statistic_ext s LEFT JOIN pg_statistic_ext_data d ON (d.stxoid = s.oid)
WHERE s.stxname = 'data1_col1_col2' ;
 stxdndistinct | stxddependencies
-----+-----
[              +| [              +
  {              +| {              +
    "ndistinct": 10,+|    "degree": 1.000000,+
    "attributes": [ +|    "attributes": [  +
      1,              +|      1              +
      2              +|    ],              +
    ]              +|    "dependency": 2  +
  }              +| }              +
]              | ]
(1 row)
```

2.5.9. ON CONFLICT Clause

In PostgreSQL 19, specifying an operator class in the INSERT ON CONFLICT statement causes an error. The following INSERT statement worked successfully in PostgreSQL 18. [\[630a937\]](#)

Example 8 Specifying the ON CONFLICT clause and operator classes

```
postgres=> INSERT INTO data1 VALUES (1, 'data1') ON CONFLICT
           (col1 INT4_OPS (fillfactor=10)) DO NOTHING ;
ERROR:  operator class options are not allowed in ON CONFLICT clause
LINE 1: INSERT INTO data1 VALUES (1, 'data1') ON CONFLICT (col1 INT4_OPS (fi...
                                     ^
```

2.5.10. RADIUS Authentication

RADIUS/UDP authentication is no longer supported. [[a1643d4](#)]

2.5.11. Command psql

The output of the \d+ metacommand for partition tables and inheritance tables has been changed. [[7bff9f1](#), [1a11405](#)]

Example 9 PostgreSQL 18 Output Formats

```
postgres=> CREATE TABLE part1(col1 INT, col2 INT) PARTITION BY LIST (col1) ;
CREATE TABLE
postgres=> CREATE TABLE part1v1 PARTITION OF part1 FOR VALUES IN (100) ;
CREATE TABLE
postgres=> \d+ part1
```

Partitioned table "public.part1"							
Column	Type	Collation	Nullable	Default	Storage	Compress	...
col1	integer				plain		...
col2	integer				plain		...

Partition key: LIST (col1)

Partitions: part1v1 FOR VALUES IN (100)

Example 10 PostgreSQL 19 Output Formats

```
postgres=> CREATE TABLE part1(col1 INT, col2 INT) PARTITION BY LIST (col1) ;
CREATE TABLE
postgres=> CREATE TABLE part1v1 PARTITION OF part1 FOR VALUES IN (100) ;
CREATE TABLE
postgres=> \d+ part1
```

Partitioned table "public.part1"

Column	Type	Collation	Nullable	Default	Storage	Compress	...
col1	integer				plain		...
col2	integer				plain		...

Partition key: LIST (col1)

Partitions:

part1v1 FOR VALUES IN (100)

2.5.12. MULE_INTERNAL Encoding

The MULT_INTERNAL character encoding has been removed. With this change, the pg_upgrade command now checks for unsupported encodings. [\[77645d4\]](#)

2.5.13. COPY FROM Statement

The COPY FROM statement that specifies a WHERE clause using a system column can no longer be executed. [\[21c69dc\]](#)

Example 11 WHERE clauses using system columns

```
postgres=# COPY data1 FROM stdin WHERE xmin = 100 ;
ERROR:  system columns are not supported in COPY FROM WHERE conditions
DETAIL:  Column "xmin" is a system column.
```

2.5.14. JSON_ARRAY Function

In PostgreSQL 18 and earlier, the JSON_ARRAY function returned NULL when an empty result set was specified. In PostgreSQL 19, the JSON_ARRAY function returns an empty JSON array, in accordance with the SQL/JSON standard. [[8d829f5](#)]

Example 12 PostgreSQL 18 Output

```
postgres=> \pset null
Null display is "NULL".
postgres=> SELECT JSON_ARRAY(SELECT i FROM (VALUES (1), (2), (NULL), (4)) foo(i)
WHERE i > 4) ;
 json_array
-----
      NULL
(1 row)
```

Example 13 PostgreSQL 19 Output

```
postgres=> SELECT JSON_ARRAY(SELECT i FROM (VALUES (1), (2), (NULL), (4)) foo(i)
WHERE i > 4) ;
 json_array
-----
      []
(1 row)
```


3. New Feature Detail

3.1. Architecture

Describes the improved architecture and features in PostgreSQL 19.

3.1.1. Changed System Catalogs

The following catalogs and views have been changed. [[228c370](#), [167ed80](#), [fd5a1a0](#), [deb6744](#), [b71bac4](#), [a5b5432](#), [a850be2](#), [185e304](#), [d3b6183](#), [96b3784](#), [f9a09aa](#), [f6a4c49](#), [3edaf29](#), [76b7872](#), [e68b6ad](#), [5db6a34](#), [ab40db3](#), [0d78952](#), [f3c9e34](#), [c257ba8](#), [a36164e](#), [307447e](#), [fb80f38](#), [ac58465](#), [723619e](#), [8fe315f](#), [2f094e7](#), [3b88e50](#), [4019f72](#), [01d485b](#), [87f61f0](#)]

Table 4 Add system catalogs and views

Catalog/View Name	Description
pg_dsm_registry_allocations	Provides information on Dynamic Shared Memory (DSM)
pg_propgraph_element	Property Graph Elements
pg_propgraph_element_label	Property Graph Element Labels
pg_propgraph_label	Property Graph Labels
pg_propgraph_label_property	Property Graph Label Properties
pg_propgraph_property	Property Graph Properties
pg_publication_sequences	Sequences including in PUBLICATION
pg_stat_autovacuum_scores	VACUUM score data for each table
pg_stat_lock	Runtime statistics for the locked condition
pg_stat_progress_data_checksums	Progress report on checksum configuration Changes
pg_stat_progress_repack	REPACK statement progress report
pg_stat_recovery	Report on the recovery status of the startup process

Table 5 System catalogs/views with additional columns

Catalog/View name	Added column	Data type	Description
pg_available_extensions	location	text	Directory for storing extension modules
pg_available_extension_versions	location	text	Directory for storing extension modules
pg_foreign_data_wrapper	fdwconnection	oid	OID for Foreign Data Wrapper connection function
pg_publication	puballsequences	boolean	Is FOR ALL SEQUENCES specified?
pg_publication_rel	prexcept	boolean	Indicates whether it is a replication exception table
pg_range	rngconstruct2	regproc	OID for a two-argument constructor
	rngconstruct3	regproc	OID for a three-argument constructor
	rngmltconstruct0	regproc	OID for a multi-range constructor with zero arguments
	rngmltconstruct1	regproc	OID for a multi-range constructor with one arguments
	rngmltconstruct2	regproc	OID for a multi-range constructor with two arguments
pg_replication_slots	slotsync_skip_reason	text	The reason for skipping slot synchronization
pg_stats	tableid	oid	OID of the table
	attnum	smallint	Number of columns
pg_stats_ext	tableid	oid	OID of the table
	statistics_id	oid	Extended statistics object OID
pg_stats_ext_exprs	tableid	oid	OID of the table
	statistics_id	oid	Extended statistics object OID
	range_length_histogram	anyarray	Histogram with non-empty bin length
	range_empty_fraction	real	Percentage of empty values
	range_bounds_histogram	anyarray	Histogram of the upper and lower limits of a range

Catalog/View name	Added column	Data type	Description
pg_stat_database_conflicts	stats_reset	timestampz	Data reset timestamp
pg_stat_progress_analyze	started_by	text	ANALYZE trigger (manual, autovacuum)
pg_stat_progress_basebackup	backup_type	text	Specify the type of backup (full or incremental)
pg_stat_progress_vacuum	mode	text	VACUUM Execution Modes (normal, aggressive, failsafe)
	started_by	text	VAUUM trigger (manual, autovacuum, autovacuum_wraparound)
pg_stat_replication_slots	mem_exceeded_count	bigint	Number of times the decoding memory usage exceeded logical_decoding_work_mem
	slotsync_skip_count	bigint	Number of synchronization skips for replication slots
	slotsync_last_skip	timestampz	Last timestamp of synchronization skip for replication slots
pg_stat_subscription_stats	confl_update_deleted	bigint	Number of times the item to be updated has been deleted in other sources
	sync_seq_error_count	bigint	Number of sequence synchronization errors
	sync_table_error_count	bigint	Number of synchronization errors for the table
pg_stat_user_functions	stats_reset	timestampz	Statistics reset timestamp
pg_stat_wal	wal_fpi_bytes	numeric	Number of bytes for full-page write
pg_subscription	subretaindeadtuples	boolean	Is update_deleted detection enabled?
	submaxretention	int	Maximum retention period for dead doubles
	subretentionactive	boolean	To keep the dead tuple?
	subserver	oid	OID of the FOREIGN SERVER

Catalog/View name	Added column	Data type	Description
			for PUBLICATION connection
	subwalrcvtimeou t	text	WAL receiver timeout
pg_stat_{all sys user}_{ta bles indexes}	stats_reset	timestampz	Statistics reset timestamp
pg_statio_{all sys user}_{ {tables indexes}}	stats_reset	timestampz	Statistics reset timestamp

Table 6 Vues within the pg_catalog schema where columns have been deleted

Catalog/View name	Remove column	Description
pg_stat_subscription_stats	sync_error_count	Renamed to sync_tale_error_count

Table 7 Views in the added information_schema schema

Catalog/View name	Description
pg_edge_table_components	Property Graph object information
pg_element_table_key_columns	Property Graph object key columns
pg_element_table_labels	Property Graph object labels
pg_element_table_properties	Property Graph object properties
pg_element_tables	Property Graph object tables
pg_labels	Property Graph object labels
pg_property_graph_privileges	Property Graph object privileges
property_graphs	Property Graph information

Table 8 System catalogs/views that had their content changed

Catalog/View name	Description
pg_aggregate	The aggtransspace column allows negative values. It is used when the transition state cannot be predicted, such as with array_agg function.
pg_class	The tuple with "relkind='g'", which indicates a PROPERTY GRAPH object, is added
pg_subscription_rel	Information about the sequences to be replicated is added
pg_stat_wal_receiver	The "connecting" message may appear in the status column

□ View pg_dsm_registry_allocations

The pg_dsm_registry_allocations view provides information about Dynamic Shared Memory (DSM). This view can only be accessed by users with the SUPERUSER privilege or who are members of the pg_read_all_stats role. [[167ed80](#)]

Table 9 Structure of the pg_dsm_registry_allocations view

Column name	Data type	Description
name	text	Entry name
type	text	Indicates the entry type. The following values are returned. • segment • area • hash
size	bigint	Entry size

Example 14 Querying the pg_dsm_registry_allocations view

```
postgres=# SELECT * FROM pg_dsm_registry_allocations ;
   name   | type   | size
-----+-----+-----
 autoprewarm | segment |    44
(1 row)
```

□ View pg_stat_lock

The pg_stat_lock view provides cumulative information on lock durations. This view is accessible to all users. [[4019f72](#)]

Table 10 Structure of the pg_stat_lock view

Column name	Data type	Description
locktype	text	Type of the lock
waits	bigint	Total number of waiting times
wait_time	bigint	Total waiting time
fastpath_exceeded	bigint	Number of times a lock could not be acquired via the fast path
stats_reset	timestamp with time zone	Timestamp of the statistics reset

Example 15 Querying of the pg_stat_lock view

```
postgres=# SELECT locktype, waits, wait_time, fastpath_exceeded FROM
           pg_stat_lock ;
```

locktype	waits	wait_time	fastpath_exceeded
relation	1	1000	0
extend	0	0	0
frozenid	0	0	0
page	0	0	0
tuple	0	0	0
transactionid	0	0	0
virtualxid	0	0	0
spectoken	0	0	0
object	0	0	0
userlock	0	0	0
advisory	0	0	0
applytransaction	0	0	0

```
(12 rows)
```

□ View pg_stat_progress_data_checksums

The pg_stat_progress_data_checksums view allows real-time monitoring of the progress of checksum activation. [[f19c0ec](#)]

Table 11 Structure of the pg_stat_progress_data_checksums view

Column name	Data type	Description	Note
pid	integer	Process ID of the worker	
datid	oid	OID of the database	
datname	name	Name of the database	
phase	text	Current progressing phase	
databases_total	bigint	The total number of databases	
databases_done	bigint	The number of processed databases	
relations_total	bigint	The number of tables	
relations_done	bigint	The number of processed tables	
blocks_total	bigint	The number of total blocks	
blocks_done	bigint	The number of processed blocks	

Example 16 Querying of the pg_stat_progress_data_checksums view

```
postgres=# SELECT * FROM pg_stat_progress_data_checksums
           WHERE datname='postgres' ;
-[ RECORD 1 ]-----+-----
pid           | 155779
datid         | 5
datname       | postgres
phase        | enabling
databases_total | 0
databases_done | 0
relations_total | 293
relations_done | 0
blocks_total  | 162163
blocks_done   | 64146
```

□ View pg_stat_progress_repack

The pg_stat_progress_repack view allows real-time monitoring of the execution status of the REPACK statements. [[ac58465](#)]

Table 12 Structure of the pg_stat_progress_repack view

Column name	Data type	Description
pid	integer	Process ID of the backend server
datid	oid	OID of the database
datname	name	Name of the database
relid	oid	OID of the targeted for REPACK statement
command	text	SQL statements (REPACK, VACUUM FULL, etc.)
phase	text	Processing phase
repack_index_relid	oid	OID of the processing index
heap_tuples_scanned	bigint	Number of tuples scanned
heap_tuples_written	bigint	Number of tuples written
heap_blks_total	bigint	Number of blocks in the table
heap_blks_scanned	bigint	Number of blocks scanned
index_rebuild_count	bigint	Number of index rebuilds

Example 17 Querying of the pg_stat_progress_repack view

```
postgres=> SELECT * FROM pg_stat_progress_repack ;
```

```
-[ RECORD 1 ]-----+-----
```

```
pid           | 53203
datid         | 5
datname       | postgres
relid         | 16390
command       | REPACK
phase         | seq scanning heap
repack_index_relid | 0
heap_tuples_scanned | 4141595
heap_tuples_inserted | 4141595
heap_tuples_updated | 0
heap_tuples_deleted | 0
heap_blks_total   | 54055
heap_blks_scanned | 22387
index_rebuild_count | 0
```


□ View `pg_stat_recovery`

The `pg_stat_recovery` view provides the recovery status of the standby database. This view can be retrieved by users with the `SUPERUSER` privilege or who have the `pg_monitor` role. [\[01d485b\]](#)

Table 13 Structure of the `pg_stat_recovery` view

Column name	Data type	Description
<code>promote_triggered</code>	boolean	Whether a promotion occurred
<code>last_replayed_read_lsn</code>	pg_lsn	Last start WAL position replayed
<code>last_replayed_end_lsn</code>	pg_lsn	Last end WAL position replayed
<code>last_replayed_tli</code>	integer	Last timeline replayed
<code>replay_end_lsn</code>	pg_lsn	WAL log location of the record currently being replayed
<code>replay_end_tli</code>	integer	Timeline of the WAL record replayed
<code>recovery_last_xact_time</code>	timestamp with time zone	Timestamp of the last transaction commit
<code>current_chunk_start_time</code>	timestamp with time zone	Time when the startup process observed
<code>pause_state</code>	text	Recovery pause state

Example 18 Querying of the `pg_stat_recovery` view

```
postgres=# SELECT * FROM pg_stat_recovery ;
-[ RECORD 1 ]-----+-----
promote_triggered    | f
last_replayed_read_lsn | 0/030005D8
last_replayed_end_lsn | 0/03000610
last_replayed_tli    | 1
replay_end_lsn       | 0/03000610
replay_end_tli       | 1
recovery_last_xact_time | 2026-06-04 14:43:09.97718+09
current_chunk_start_time | 2026-06-04 14:45:43.83114+09
pause_state          | not paused
```

3.1.2. Logical Replication Enhancements

The following new features have been implemented for logical replication

□ Sequence Replication

Sequence replication is now supported in logical replication. On the publication side, create the publication by specifying the ALL SEQUENCES clause. The names of the sequences contained in the publication are stored in the pg_publication_sequences view. [\[5509055\]](#)

Example 19 Creating a publication that includes a sequence

```
postgres=# CREATE PUBLICATION pub1 FOR ALL SEQUENCES ;
CREATE PUBLICATION
postgres=# SELECT * FROM pg_publication_sequences ;
  pubname | schemaname | sequencename
-----+-----+-----
  pub1    | public     | seq01
(1 row)
```

On the subscriber side, create a subscription after first creating the sequences to be replicated. The target sequences are stored in the pg_subscription_rel view.

Example 20 Create a subscription

```
postgres=# CREATE SUBSCRIPTION sub1 CONNECTION
           'host=dbsvr1 port=5432 dbname=postgres' PUBLICATION pub1 ;
NOTICE:  created replication slot "sub1" on publisher
CREATE SUBSCRIPTION
postgres=# SELECT * FROM pg_subscription_rel ;
 ssubid | srrelid | srsubstate | srsublsn
-----+-----+-----+-----
  16390 |  16389 | r          | 0/030659C0
(1 row)
```

To synchronize sequence values, execute the ALTER SUBSCRIPTION REFRESH SEQUENCES statement on the subscriber side.

Example 21 Synchronization of sequence values

```
postgres=# SELECT setval('seq01', 1) ;
      setval
-----
         1
(1 row)

postgres=# ALTER SUBSCRIPTION sub1 REFRESH SEQUENCES ;
ALTER SUBSCRIPTION
postgres=# SELECT nextval('seq01') ;
      nextval
-----
         11
(1 row)
```

□ SERVER clause in the CREATE SUBSCRIPTION statement

Previously, the connection information for connecting to the PUBLICATION was specified directly in the CONNECTION clause of the CREATE SUBSCRIPTION statement. In PostgreSQL 19, it is now possible to specify an existing SERVER object in the SERVER clause. [\[8185bb5\]](#)

Example 22 Specifying the SERVER clause

```
postgres=# CREATE EXTENSION postgres_fdw ;
CREATE EXTENSION
postgres=# CREATE SERVER dbsvr1 FOREIGN DATA WRAPPER postgres_fdw OPTIONS
          (host 'dbsvr1', port '5432', dbname 'postgres') ;
CREATE SERVER
postgres=# CREATE USER MAPPING FOR PUBLIC SERVER dbsvr1 OPTIONS
          (user 'postgres', password '<<Password>>') ;
CREATE USER MAPPING
postgres=# CREATE SUBSCRIPTION sub1 SERVER dbsvr1 PUBLICATION pub1 ;
NOTICE:  created replication slot "sub1" on publisher
CREATE SUBSCRIPTION
```

□ Specifying Exception Tables

The EXCEPT clause can be specified for the PUBLICATION that uses the ALL TABLES clause to designate tables to be excepted. The EXCEPT clause is specified in the CREATE PUBLICATION or ALTER PUBLICATION statement. The preexcept column has been added to the pg_publication_rel catalog to indicate exception tables. [[5984ea8](#), [fd36606](#), [493f8c6](#)]

Example 23 Specifying the exception table

```
postgres=# CREATE PUBLICATION pub1 FOR ALL TABLES EXCEPT (TABLE extable1) ;
CREATE PUBLICATION
postgres=# SELECT * FROM pg_publication_rel ;
   oid   | prpubid | prrelid | prexcept | prqual | prattrs
-----+-----+-----+-----+-----+-----
 16414   |  16413 |  16402 | t        |        |
(1 row)
```

□ Subscription option retain_dead_tuples

The retain_dead_tuples option has been added to subscriptions. Enabling this option causes the subscriber to retain deleted tuples until the remote transaction is completed. When setting the retain_dead_tuples option to "true", the track_commit_timestamp parameter must also be set to "true" (default: "false"). [[228c370](#)]

Example 24 Setting the retain_dead_tuples option

```
postgres=# CREATE SUBSCRIPTION sub1 CONNECTION 'port=5432 dbname=postgres'
          PUBLICATION pub1 WITH (RETAIN_DEAD_TUPLES = TRUE) ;
NOTICE:  created replication slot "sub1" on publisher
CREATE SUBSCRIPTION
postgres=# SELECT subname, subretaindeadtuples FROM pg_subscription ;
-[ RECORD 1 ]-----+-----
subname          | sub1
subretaindeadtuples | t
```

□ Subscription option max_retention_duration

The max_retention_duration option has been added to subscriptions. This option sets the maximum retention period for non-deletable transaction IDs in environments where the retain_dead_tuples

option is enabled. Once this threshold is exceeded, the apply worker stops retaining conflict detection information. The setting for this option can be verified in the submaxretention column of the pg_subscription catalog. [\[a850be2\]](#)

Example 25 Setting the max_retention_duration option

```
postgres=# CREATE SUBSCRIPTION sub1 CONNECTION 'port=5432 dbname=postgres'
PUBLICATION pub1 WITH (MAX_RETENTION_DURATION = 10, RETAIN_DEAD_TUPLES =
TRUE) ;
NOTICE:  created replication slot "sub1" on publisher
CREATE SUBSCRIPTION
postgres=# SELECT subname, submaxretention, subretentionactive FROM
           pg_subscription ;
 subname | submaxretention | subretentionactive
-----+-----+-----
 sub1    |          10    | t
(1 row)
```

□ Detection of Memory Usage for Decoding

The mem_exceeded_count column has been added to the pg_stat_replication_slots view. This column indicates the number of times the memory used for logical decoding exceeded the logical_decoding_work_mem parameter. [\[d3b6183\]](#)

Example 26 Querying the pg_stat_replication_slots view

```
postgres=# SELECT slot_name, mem_exceeded_count FROM
           pg_stat_replication_slots ;
-[ RECORD 1 ]-----+-----
slot_name      | sub1
mem_exceeded_count | 0
```

□ Conflict Detection

The update_deleted conflict can now be detected when an update on a subscriber conflicts with a local deletion. To use this feature, the retain_dead_tuples option must be enabled on the subscriber. [\[fd5a1a0\]](#)

□ Automatic WAL Level Change

Even if the `wal_level` parameter is set to its default value of "replica", the WAL output level is automatically changed to "logical" when a logical replication slot is created. The internal parameter `effective_wal_level` has been added to indicate this state. When all logical replication slots are deleted, the value of the `effective_wal_level` parameter reverts to its original state. [[67c2097](#)]

Example 27 Automatic Update of WAL Output Level

```
postgres=# SHOW wal_level ;
wal_level
-----
replica
(1 row)

postgres=# SHOW effective_wal_level ;
effective_wal_level
-----
replica
(1 row)

postgres=# SELECT
            pg_create_logical_replication_slot('slot1', 'test_decoding') ;
 slot_name |      lsn
-----+-----
 slot1     | 0/95033BD8
(1 row)

postgres=# SHOW effective_wal_level ;
effective_wal_level
-----
logical
(1 row)
```

3.1.3. Partition Table Enhancements

The following new features have been implemented in the partition table.

□ Merging Partitions

It is now possible to merge multiple partitions into a single partition using an ALTER TABLE statement on RANGE or LIST partitioned tables. Specify the existing partitions in the MERGE PARTITIONS clause and specify the name of the new partition to be created as the destination in the INTO clause. [[f2e4cc4](#), [713e553](#)]

Syntax 1 MERGE PARTITIONS clause

```
ALTER TABLE table_name MERGE PARTITIONS (partition#1, partition#2, ...)
        INTO new_partition
```

The following example merges partitions part1v1 and part1v2 into the part1v12 partition.

Example 28 Merging Partitions

```
postgres=> CREATE TABLE part1(c1 INT PRIMARY KEY, c2 VARCHAR(10))
           PARTITION BY RANGE(c1) ;
CREATE TABLE
postgres=> CREATE TABLE part1v1 PARTITION OF part1 FOR VALUES
           FROM (0) TO (1000000) ;
CREATE TABLE
postgres=> CREATE TABLE part1v2 PARTITION OF part1 FOR VALUES
           FROM (1000000) TO (2000000) ;
CREATE TABLE
postgres=> INSERT INTO part1 VALUES (generate_series(1, 1999999), 'data1') ;
INSERT 0 1999999
postgres=> ALTER TABLE part1 MERGE PARTITIONS (part1v1, part1v2) INTO part1v12 ;
ALTER TABLE
```

RANGE partitions whose data ranges are not contiguous cannot be merged.

Example 29 Partition merge failed

```
postgres=> CREATE TABLE part2(c1 INT PRIMARY KEY, c2 VARCHAR(10))
           PARTITION BY RANGE(c1) ;
CREATE TABLE
postgres=> CREATE TABLE part2v1 PARTITION OF part2 FOR VALUES
           FROM (0) TO (1000000) ;
CREATE TABLE
postgres=> CREATE TABLE part2v2 PARTITION OF part2 FOR VALUES
           FROM (1000000) TO (2000000) ;
CREATE TABLE
postgres=> CREATE TABLE part2v3 PARTITION OF part2
           FOR VALUES FROM (2000000) TO (3000000) ;
CREATE TABLE
postgres=> ALTER TABLE part2 MERGE PARTITIONS (part2v1, part2v3) INTO part2v13 ;
ERROR:  cannot merge partition "part2v3" together with partition "part2v1"
DETAIL:  The lower bound of partition "part2v3" is not equal to the upper bound
of partition "part2v1"
HINT:   ALTER TABLE ... MERGE PARTITIONS requires the partition bounds to be
adjacent.
```

□ Partition Splitting

It is now possible to split a partition into multiple partitions using the ALTER TABLE statement on RANGE/LIST partition tables. Specify the source partition in the SPLIT PARTITION clause of the ALTER TABLE statement, and specify the destination partitions and partitioning values in the INTO clause. [[4b3d173](#), [713e553](#)]

Syntax 2 SPLIT PARTITION clause

```
ALTER TABLE table_name SPLIT PARTITION old_partition INTO
(PARTITION partition#1 VALUES ...,
 PARTITION partiotn#2 VALUES ...)
```


Example 30 Splitting the partition

```
postgres=> CREATE TABLE part3(c1 INT PRIMARY KEY, c2 VARCHAR(10))
           PARTITION BY RANGE(c1) ;

CREATE TABLE
postgres=> CREATE TABLE part3v1 PARTITION OF part3 FOR VALUES
           FROM (0) TO (2000000) ;

CREATE TABLE
postgres=> INSERT INTO part3 VALUES (generate_series(1,1999999), 'data1') ;
INSERT 0 1999999

postgres=> ALTER TABLE part3 SPLIT PARTITION part3v1 INTO
           (PARTITION part3v2 FOR VALUES FROM (0) TO (1000000),
            PARTITION part3v3 FOR VALUES FROM (1000000) TO (2000000)) ;

ALTER TABLE
```

3.1.4. SQL Property Graph

PostgreSQL 19 introduces the PROPERTY GRAPH object, which provides graph query features. This feature implements Property Graph Queries (SQL/PgQ) - ISO/IEC 9075-16:2023. PROPERTY GRAPH objects are created using the CREATE PROPERTY GRAPH statement. It creates a table that serves as the source for graph queries and maps it to a PROPERTY GRAPH. VERTEX and NODE, as well as EDGE and RELATIONSHIP, have the same meanings. This document does not provide detailed instructions on how to use it. [\[2f094e7\]](#)

Syntax 3 Creating a Property Graph

```
CREATE PROPERTY GRAPH name
  [{VERTEX|NODE} TABLES (vertex_table_definition [, ...])]
  [{EDGE|RELATIONSHIP} TABLES (edge_table_definition [, ...])]
```

Example 31 Creating a Property Graph

```
postgres=> CREATE TABLE member (member_id INT PRIMARY KEY, member_name TEXT) ;
CREATE TABLE
postgres=> CREATE TABLE follow (member_id1 INT, member_id2 INTEGER,
PRIMARY KEY(member_id1, member_id2)) ;
CREATE TABLE
postgres=> CREATE PROPERTY GRAPH graph1
  VERTEX TABLES (member KEY (member_id) PROPERTIES (member_id, member_name))
  EDGE TABLES (follow
    SOURCE KEY (member_id1) REFERENCES member (member_id)
    DESTINATION KEY (member_id2) REFERENCES member (member_id)) ;
CREATE PROPERTY GRAPH
```

Several catalogs with names beginning with "pg_prograph" have been added for the Property Graph. Use the GRAPH_TABLE function to query the Property Graph.

Example 32 Querying PROPERTY GRAPH

```
postgres=> SELECT * FROM GRAPH_TABLE (graph1 MATCH
  (mem1 IS member)-[follow1 IS follow]->(mem2 IS member)
  -[follow2 IS follow]->(mem3 IS member)
  WHERE mem1.member_id = mem3.member_id COLUMNS
    (mem1.member_id, mem2.member_id)) ;
member_id | member_id
-----+-----
      100 |       200
      200 |       100
(2 rows)
```

3.1.5. SSL/SNI

PostgreSQL now supports SSL Server Name Indication (SNI) on the server side. This feature is enabled by setting the ssl_sni parameter to "on" (the default value is "off"). To configure SNI, use the pg_hosts.conf file in the database cluster. In the pg_hosts.conf file, enter the following entries, one per line. [\[4f43302\]](#)

Table 14 Contents of the pg_hosts.conf file

Setting	Description	Note
HOSTNAME	Hostname (required)	
SSL CERTIFICATE	SSL certificate file (required)	
SSL KEY	SSL private key file (required)	
SSL CA	SSL authentication file (optional)	
PASSPHRASE COMMAND	Passphrase (optional)	
PASSPHRASE COMMAND RELOAD	Reload passphrase (optional)	

3.1.6. Dynamic Checksum Configuration

It is now possible to enable or disable block checksums while the instance is running. To enable checksums, execute the `pg_enable_data_checksums` function; to disable them, execute the `pg_disable_data_checksums` function. Enabling checksums is executed by a background worker process. When enabling checksums, the `pg_enable_data_checksums` function allows specifying the `cost_delay` parameter (default value 0) and the `cost_limit` parameter (default value 100), which enables checksums in a behavior similar to VACUUM cost delay. Checksum enablement executes in the background, and progress can be tracked via the `pg_stat_progress_data_checksums` view. [[f19c0ec](#)]

Example 33 Dynamic Modification of Checksum Settings

```
postgres=# SHOW data_checksums ;
-[ RECORD 1 ]--+-
data_checksums | on

postgres=# SELECT pg_disable_data_checksums() ;
-[ RECORD 1 ]-----+
pg_disable_data_checksums |

postgres=# SHOW data_checksums ;
-[ RECORD 1 ]--+-
data_checksums | off

postgres=# SELECT pg_enable_data_checksums(1, 110) ;
-[ RECORD 1 ]-----+
pg_enable_data_checksums |

postgres=# SHOW data_checksums ;
-[ RECORD 1 ]--+-
data_checksums | inprogress-on
```

3.1.7. Roles

The `pg_read_all_data` and `pg_write_all_data` functions now support reading and writing large objects. In previous versions, the error "ERROR: permission denied for large object {OID}" occurred. [\[d981976\]](#)

Example 34 Permissions for large objects

```
postgres=# SELECT lo_create(-1) ;
 lo_create
-----
 4294967295
(1 row)

postgres=# SELECT lo_put(4294967295, 1, E'\\xaa') ;
 lo_put
-----

(1 row)

postgres=# GRANT pg_read_all_data TO user1 ;
GRANT ROLE
postgres=# \connect postgres user1
You are now connected to database "postgres" as user "user1".
postgres=> SELECT lo_get(4294967295) ;
 lo_get
-----
 \x00aa
(1 row)
```

3.1.8. Libpq

The following APIs have been added or enhanced.

□ Connection string servicefile

The servicefile option has been added to the connection string to specify a service file. The PGSERVICEFILE environment variable can still be specified as previously. [\[092f3c6\]](#)

Example 35 Connection string servicefile

```
$ psql "servicefile=/tmp/serviceA.conf service=service1"
psql (19beta1)
Type "help" for help.

postgres=#
```

□ Connection string oauth_ca_file

The `oauth_ca_file` option has been added to the connection string to specify an OAuth certificate. The `PGOAUTHCAFILE` environment variable is now available in non-debug modes as well. [\[9933681\]](#)

□ Retrieving Version Information

The `get_pg_version` function, which retrieves version information from the `PG_VERSION` file, has been added. This function is used by commands such as `pg_resetwal`, `pg_combinebackup`, `pg_createsubscriber`, and `pg_upgrade`. [\[cd0be13\]](#)

□ Other API

The following APIs have been added. [\[fe07100\]](#), [7508169](#), [e533524](#), [b8d7685](#)

Table 15 Added API names

API name	Description	Note
<code>pg_database_locale</code>	Retrieve the database's default locale	
<code>GetNamedDSA</code>	Assign the DSA	
<code>GetNamedDSHash</code>	Assign the DSA hash	
<code>ParallelSlotSetIdle</code>	Mark <code>ParallelSlot</code> as idle	
<code>PQgetThreadLock</code>	Retrieve lock information for the current thread	

3.1.9. I/O Processing

These are the improvements to I/O processing in PostgreSQL 19.

□ Automatic Scaling of Asynchronous I/O Workers

The number of I/O workers now automatically increases or decreases in response to load. The number of active I/O workers is determined by the following parameters: As the I/O queue grows, the number of workers increases; if a worker remains idle for a certain period of time, it is stopped. [\[d1c01b7\]](#)

Table 16 Parameters related to I/O workers

Parameter name	Description	Default value
io_min_workers	Minimum number of I/O workers	2
io_max_workers	Maximum number of I/O workers	8
io_worker_idle_timeout	Idle time before a worker terminates	60s
io_worker_launch_interval	Interval between worker launches	100ms

□ Enhanced Streaming I/O

PostgreSQL 19 has enhanced the scope of streaming I/O.

- VACUUM for BRIN index [\[a6eac22\]](#)
- Data deleting reads for hash index [\[bfa3c4f\]](#)
- Index reads in the Contrib module bloom [\[4c910f3\]](#), [\[d841ca2\]](#)
- Index reads in the Contrib module pgstattuple [\[213f007\]](#)

□ Improvements to asynchronous I/O processing

When using `io_uring` with I/O methods, the behavior has been changed so that synchronous I/O is used for reads of four blocks or fewer. [\[a9ee668\]](#), [\[8ca147d\]](#), [\[f63ca33\]](#)

3.1.10. Automatic Vacuum

The following features have been added to the automatic VACUUM processing.

□ Using Parallel Workers

It is now possible to use parallel workers during automatic VACUUM operations. Automatic VACUUM performs the same processing as the VACUUM (PARALLEL) statement. The number of parallel workers is determined by the `autovacuum_max_parallel_workers` parameter. The default value for this parameter is 0. Refer to the automatic VACUUM log to verify that parallel workers are launched.

Example 36 Automatic VACUUM Log

```
LOG:  automatic vacuum of table "postgres.public.data1": index scans: 1
...
index scan needed: 5406 pages from table (100.00% of total) had 500000 dead ...
parallel workers: index vacuum: 1 planned, 1 launched in total
...
```

□ Prioritization by scoring

PostgreSQL 19 introduced the scoring system that determines the execution order of automatic VACUUM operations. It prioritizes tables that deviate the most from the thresholds used to trigger VACUUM and ANALYZE operations. Scores are determined by the newly added `pg_stat_autovacuum_scores` view. For example, if 100 tuples are updated by an UPDATE statement and the VACUUM threshold is 80, the score will be 1.25. The actual score is determined by multiplying this value by the following parameters, which represent their respective weights. [\[87f61f0\]](#)

Table 17 Parameters related to the score

Parameter name	Default value	Description
<code>autovacuum_analyze_score_weight</code>	1.0	ANALYZE score weight
<code>autovacuum_freeze_score_weight</code>	1.0	FREEZE score weight
<code>autovacuum_multixact_freeze_score_weight</code>	1.0	Multi-Transaction FREEZE score weight
<code>autovacuum_vacuum_insert_score_weight</code>	1.0	VACUUM INSRT score weight
<code>autovacuum_vacuum_score_weight</code>	1.0	VACUUM score weight

The structure of the `pg_stat_autovacuum_scores` view is as follows.

Table 18 Structure of the pg_stat_autovacuum_scores view

Column name	Data type	Description
relid	oid	OID of the table
schemaname	name	Name of the schema
relname	name	Name of the table
score	double precision	Maximum value of all component scores
xid_score	double precision	Transaction ID age score
vacuum_score	double precision	Vacuum score
vacuum_insert_score	double precision	Vacuum insert score
analyze_score	double precision	Analyze score
do_vacuum	boolean	Whether autovacuum would vacuum the table
do_analyze	boolean	Whether autovacuum would analyze the table
for_wraparound	boolean	Whether autovacuum would vacuum the table for wraparound prevention

Example 37 Querying the pg_stat_autovacuum_scores view

```
postgres=> SELECT * FROM pg_stat_autovacuum_scores WHERE relname='data1' ;
-[ RECORD 1 ]-----+-----
relid          | 16385
schemaname     | public
relname        | data1
score          | 9.990009990000999
xid_score      | 2e-08
mxid_score     | 0
vacuum_score   | 4.997501249375312
vacuum_insert_score | 0
analyze_score  | 9.990009990000999
do_vacuum      | t
do_analyze     | t
for_wraparound | f
```

3.1.11. Hook

The following hooks have been added for the planner.

- Hook `planner_setup_hook`, `planner_shutdown_hook`

The `planner_setup_hook` hook, which is executed when the `PlannerGlobal` object is initialized, and the `planner_shutdown_hook` hook, which is executed immediately before the object is destroyed, have been added. [[94f3ad3](#), [0f4c8d3](#)]

- Hook `joinrel_setup_hook`, `join_path_setup_hook`

The `joinrel_setup_hook` hook, which is executed when creating table join relations (`joinrel`), and the `join_path_setup_hook` hook, which is executed when adding paths to a `joinrel`, have been added. [[4020b37](#)]

3.1.12. Faster processing

Performance improvements have been implemented through changes to processor instructions and sorting algorithms.

- Sorting

Some sorting algorithms have been changed from Quick sort to Radix sort. If there are fewer than 40 items, Quick sort will be used. [[ef3c3cf](#)]

- Internal functions `hex_encode` / `hex_decode`

The `hex_encode` and `hex_decode` functions now support SIMD instructions for the x86-64 and Aarch64 architectures. The larger the input value, the more effective the performance improvement is expected to be. [[ec8719c](#)]

- Page Checksum

If available, AVX2 instructions are used to calculate the page checksum. [[5e13b0f](#)]

- CRC32C

If the Crypto Extension is available on an Arm v8 processor, it is used for CRC32C calculations. [[fbc57f2](#)]

- COPY FROM statement

Performance has been improved by using SIMD instructions to check for the presence of special characters. [[e0a3a3f](#)]

3.1.13. Log Output

The byte count for full-page images is now included in the logs generated by the automatic ANALYZE operation. [[ad25744](#)]

Example 38 Automatic ANALYZE Log

```
LOG:  automatic analyze of table "postgres.public.data1"
      avg read rate: 1.486 MB/s, avg write rate: 0.255 MB/s
      buffer usage: 10994 hits, 35 reads, 6 dirtied
      WAL usage: 14 records, 3 full page images, 27293 bytes, 24576 full page
      image bytes, 0 buffers full
      system usage: CPU: user: 0.03 s, system: 0.01 s, elapsed: 0.18 s
```

3.1.14. OAuth Authentication

The following new features have been added to the OAuth authentication feature.

□ File pg_hba.conf Settings

It is now possible to specify OAuth validator information in the pg_hba.conf options. The following is an example of pg_hba.conf from the manual. [[b977bd3](#)]

Example 39 OAuth settings in the pg_hba.conf file

```
hostssl postgres admin 0.0.0.0/0 oauth issuer=https://admin.example.com \
                                scope="pg-admin openid email" \
                                map=oauth-email \
                                validator=my_validator \
                                validator.foo="admin access" \
                                validator.bar=https://magic.example.com
```

□ Validator Error Output

The following parameters have been added to control how errors generated by the validator are logged. [[d438a36](#)]

Table 19 Added parameters

Parameter name	Description	Default value
oauth_validator.error_detail	Error information contained in DETAIL	"
oauth_validator.internal_error	Whether to output internal errors	false

3.1.15. Wait Events

The following wait events have been added. [[6c5c393](#), [e05a24c](#), [7a39f43](#)]

Table 20 Added Wait Events

Wait event name	Description	Note
WaitEventBuffer	Waiting for the data buffer	Changed from WaitEventBufferPin
COPY_FROM_READ	Waiting for the COPY FROM read	
COPY_TO_WRITE	Waiting for the COPY TO write	
WAIT_FOR_WAL_WRITE	Waiting to write to the standby	

3.2. SQL Statement

This section explains new features related to SQL statements.

3.2.1. ALTER DOMAIN statement

The lock level required to execute the ALTER DOMAIN VALIDATE CONSTRAINT statement has been changed from SHARE to SHARE UPDATE EXCLUSIVE. [[16a0039](#)]

3.2.2. ALTER FOREIGN DATA WRAPPER statement

The CONNECTION clause, which specifies the function used to establish a new connection, has been added. [[8185bb5](#)]

Syntax 4 ALTER FOREIGN DATA WAPPER CONNECTION statement

```
ALTER FOREIGN DATA WAPPER fdw_name CONNECTION function_name
ALTER FOREIGN DATA WAPPER fdw_name NO CONNECTION
```

3.2.3. ALTER PUBLICATION statement

It is now possible to specify the SET ALL TABLES clause and the SET ALL SEQUENCES clause in an ALTER PUBLICATION statement. This is only possible for publications that specify the ALL TABLES/SEQUENCES clause or for empty publications. [[5984ea8](#), [fd36606](#), [493f8c6](#), [5509055](#)]

Syntax 5 ALTER PUBLICATION statement

```
ALTER PUBLICATION publication_name SET ALL TABLES [EXCEPT (TABLE expression)]
ALTER PUBLICATION publication_name SET ALL SEQUENCES
```

Example 40 Executing the ALTER PUBLICATION statement

```
postgres=# CREATE PUBLICATION pub1 FOR ALL TABLES ;  
CREATE PUBLICATION  
postgres=# ALTER PUBLICATION pub1 SET ALL TABLES EXCEPT (TABLE data2) ;  
ALTER PUBLICATION  
postgres=# CREATE PUBLICATION pub2 ;  
CREATE PUBLICATION  
postgres=# ALTER PUBLICATION pub2 SET ALL SEQUENCES ;  
ALTER PUBLICATION
```

3.2.4. ALTER SUBSCRIPTION statement

The ALTER SUBSCRIPTION REFRESH SEQUENCES statement, which updates the sequence information included in the subscription, has been added. [[f0b3573](#)]

Syntax 6 ALTER SUBSCRIPTION REFRESH SEQUENCES statement

```
ALTER SUBSCRIPTION subscription_name REFRESH SEQUENCES
```

Example 41 Specifying the REFRESH SEQUENCES clause

```
postgres=# ALTER SUBSCRIPTION sub1 REFRESH SEQUENCES ;  
ALTER SUBSCRIPTION
```

3.2.5. ALTER TABLE statement

The following enhancements have been added to the ALTER TABLE statement.

□ Changing the Expression for Virtual Generated Columns

It is now possible to change the expression for virtual generated columns with a CHECK constraint using the SET EXPRESSION clause. [[f80bedd](#)]

Syntax 7 SET EXPRESSION clause

```
ALTER TABLE table_name ALTER COLUMN column_name SET EXPRESSION AS expression
```

Example 42 Altering Virtual Columns with CHECK Constraints

```
postgres=> CREATE TABLE data1(col1 INT, col2 INT GENERATED ALWAYS AS
              (col1 * 100) CHECK (col2 < 10000)) ;

CREATE TABLE
postgres=> ALTER TABLE data1 ALTER COLUMN col2 SET EXPRESSION AS
              (col1 * 200) ;

ALTER TABLE
```

□ Setting the CHECK constraint to ENFORCED

It is now possible to change the setting of the ENFORCED clause for CHECK constraints. [[342051d](#)]

Syntax 8 ENFORCED clause

```
ALTER TABLE table_name ALTER CONSTRAINT constraint_name [NOT] ENFORCED
```

Example 43 Setting the ENFORCED clause for CHECK constraints

```
postgres=> CREATE TABLE data1(col1 INT PRIMARY KEY, col2 INT) ;

CREATE TABLE
postgres=> ALTER TABLE data1 ADD CONSTRAINT data1_col2_check1
              CHECK(col2 < 1000) ;

ALTER TABLE
postgres=> INSERT INTO data1 SELECT g1, g1 FROM generate_series(1, 999) g1 ;
INSERT 0 999

postgres=> ALTER TABLE data1 ALTER CONSTRAINT data1_col2_check1 NOT ENFORCED ;

ALTER TABLE
postgres=> INSERT INTO data1 VALUES (1000, 1000) ;
INSERT 0 1

postgres=> ALTER TABLE data1 ALTER CONSTRAINT data1_col2_check1 ENFORCED ;
ERROR:  check constraint "data1_col2_check1" of relation "data1" is violated by
some row

postgres=> DELETE FROM data1 WHERE col2=1000 ;
DELETE 1

postgres=> ALTER TABLE data1 ALTER CONSTRAINT data1_col2_check1 ENFORCED ;

ALTER TABLE
```

□ Option `autovacuum_parallel_workers`

The `autovacuum_parallel_workers` option has been added for tables. This option can also be specified in the `CREATE TABLE` statement. It specifies the number of parallel `VACUUM` worker processes. The default value for this option is `-1`, and it inherits the value from the `autovacuum_max_parallel_workers` parameter. If this option is set to `0`, parallel `VACUUM` is disabled. [\[1ff3180\]](#)

Example 44 Configuring the number of parallel workers for automatic `VACUUM`

```
postgres=> ALTER TABLE data1 SET (autovacuum_parallel_workers = 4) ;
ALTER TABLE
```

□ Optimizing `ALTER TABLE` Statements

When using the `ALTER TABLE ADD COLUMN` statement, the addition of a column with a `CHECK` constraint or a `NOT NULL` constraint on a domain can now be checked for the need to rewrite the table. If no rewrite is necessary, the table will not be rewritten. [\[a0b6ef2\]](#)

3.2.6. `CHECKPOINT` statement

The following options have been added to the `CHECKPOINT` statement.

□ `MODE` option

The `MODE` option determines how checkpoints are executed. The `MODE FAST` option is the default behavior and executes checkpoints as quickly as possible. If the `MODE SPREAD` option is specified, checkpoints are executed with reduced load based on relevant parameters. [\[2f698d7\]](#)

Syntax 9 `CHECKPOINT` statement

```
CHECKPOINT [(MODE {FAST|SPREAD})]
```

Example 45 `MODE` option

```
postgres=# CHECKPOINT (MODE FAST) ;
CHECKPOINT
postgres=# CHECKPOINT (MODE SPREAD) ;
CHECKPOINT
```

Depending on the options specified, the output log will change as follows.

Table 21 Output logs

Option	Log message	Note
FAST	LOG: checkpoint starting: fast force wait	
SPREAD	LOG: checkpoint starting: force wait	

□ FLUSH_UNLOGGED option

Enabling the FLUSH_UNLOGGED option causes the database to also flush dirty buffers that have not been logged at the checkpoint. [[8d33fba](#), [a4f1265](#)]

Syntax 10 FLUSH_UNLOGGED option

```
CHECKPOINT [(FLUSH_UNLOGGED [boolean])]
```

Example 46 FLUSH_UNLOGGED option

```
postgres=# CHECKPOINT (FLUSH_UNLOGGED) ;
CHECKPOINT
```

Depending on the options specified, the output log will change as follows.

Table 22 Output logs

Option	Log message
No specification	LOG: checkpoint starting: fast force wait
FLUSH_UNLOGGED	LOG: checkpoint starting: fast force wait flush-unlogged

3.2.7. COPY statement

The following new features have been implemented to the COPY statement.

□ ON_ERROR option setting

It is now possible to specify the "set_null" for the on_error option in the COPY FROM statement. This option sets the column value to NULL if a data type conversion error occurs. [[2a525cc](#)]

Example 47 ON_ERROR option

```
postgres=# \d data1
               Table "public.data1"
  Column | Type   | Collation | Nullable | Default
-----+-----+-----+-----+-----
 col1    | integer |           |          |
 col2    | integer |           |          |

postgres=# COPY data1 FROM stdin WITH (format 'csv', on_error 'set_null') ;
Enter data to be copied followed by a newline.
End with a backslash and a period on a line by itself, or an EOF signal.
>> 100,100
>> 200,bad_data
>> 300,300
>> \.
NOTICE:  in 1 row, columns were set to null due to data type incompatibility
COPY 3
postgres=# SELECT * FROM data1 ;
 col1 | col2
-----+-----
   100 |   100
   200 | null
   300 |   300
(3 rows)
```

□ Partition Table Support

The COPY TO statement can now be executed on partition tables. In previous versions, it was necessary to execute the COPY (SELECT ...) TO statement instead of using the partition table name.

[\[4bca91f\]](#)

Example 48 Specifying the partition table

```
postgres=# \d part1
                Partitioned table "public.part1"
  Column |          Type          | Collation | Nullable | Default
-----+-----+-----+-----+-----
  col    | integer                |           | not null |
  col2   | character varying(10)  |           |         |
Partition key: RANGE (c1)
Indexes:
    "part1_pkey" PRIMARY KEY, btree (col1)
Number of partitions: 2 (Use \d+ to list them.)

postgres=# COPY part1 TO '/tmp/part1.csv' CSV ;
COPY 2000000
```

□ Number of header rows

It is now possible to specify the number of header rows in the data. [[bc2f348](#)]

Example 49 Specifying the number of header rows

```
postgres=# COPY data1 FROM stdin WITH (FORMAT CSV, HEADER 2) ;
Enter data to be copied followed by a newline.
End with a backslash and a period on a line by itself, or an EOF signal.
>> HEADER1
>> HEADER2
>> 100,data1
>> 200,data2
>> \.
COPY 2
```

□ JSON format output

It is now possible to specify "json" as the FORMAT option for the COPY TO statement. The additional option "force_array" can also be specified. [[7dadd38](#), [4c0390a](#)]

Example 50 Output to JSON format

```
postgres=# COPY data1 TO stdout WITH (FORMAT json) ;
{"col1":1,"col2":"data1"}
{"col1":2,"col2":"data2"}
{"col1":3,"col2":"data3"}
{"col1":4,"col2":"data4"}
{"col1":5,"col2":"data5"}
postgres=# COPY data1 TO stdout WITH (FORMAT json, FORCE_ARRAY) ;
[
  {"col1":1,"col2":"data1"}
, {"col1":2,"col2":"data2"}
, {"col1":3,"col2":"data3"}
, {"col1":4,"col2":"data4"}
, {"col1":5,"col2":"data5"}
]
```

3.2.8. CREATE PUBLICATION statement

It is now possible to specify the FOR ALL SEQUENCES clause, which applies to all sequences. This clause requires a user with the SUPERUSER privilege to execute the CREATE PUBLICATION statement. To indicate whether this clause has been specified, the puballsequences column has been added to the pg_publication catalog. [\[96b3784\]](#)

Syntax 11 FOR ALL SEQUENCES clause

```
CREATE PUBLICATION publication_name FOR ALL SEQUENCES
```

Example 51 Specifying the FOR ALL SEQUENCES clause

```
postgres=# CREATE PUBLICATION pub1 FOR ALL SEQUENCES ;
```

```
CREATE PUBLICATION
```

```
postgres=# \dRp
```

List of publications						
Name	Owner	All tables	All sequences	Inserts	Updates	Deletes...
pub1	postgres	f	<u>t</u>	t	t	t ...

(1 row)

No tables can be added to a publication that specifies the ALL SEQUENCES clause.

Example 52 Add a table to the publication

```
postgres=# ALTER PUBLICATION pub1 ADD TABLE data1 ;
```

```
ERROR: publication "pub1" is defined as FOR ALL SEQUENCES
```

```
DETAIL: Tables or sequences cannot be added to or dropped from FOR ALL SEQUENCES  
publications.
```

3.2.9. CREATE SCHEMA statement

It is now possible to specify objects other than tables, views, indexes, sequences, triggers, and GRANT in the CREATE SCHEMA statement. [[a9c350d](#), [404db8f](#)]

Example 53 The CREATE SCHEMA Statement and Creating Objects

```
postgres=# CREATE SCHEMA schema1
```

```
CREATE TABLE data1(col1 INT PRIMARY KEY, col2 VARCHAR(10))
```

```
CREATE COLLATION col1 (LOCALE="C")
```

```
CREATE DOMAIN d1 AS INT ;
```

```
CREATE SCHEMA
```

The following CREATE clauses can be specified:

Table 23 Types of objects that can be created at the same time

Object	PostgreSQL 18	PostgreSQL 19	Note
AGGREGATE	No	Yes	
COLLATION	No	Yes	
DOMAIN	No	Yes	
FUNCTION	No	Yes	
INDEX	Yes	Yes	
OPERATOR	No	Yes	
PROCEDURE	No	Yes	
SEQUENCE	Yes	Yes	
TABLE	Yes	Yes	
TEXT SEARCH CONFIGURATION	No	Yes	
TEXT SEARCH DICTIONARY	No	Yes	
TEXT SEARCH PARSER	No	Yes	
TEXT SEARCH TEMPLATE	No	Yes	
TRIGGER	Yes	Yes	
TYPE	No	Yes	
VIEW	Yes	Yes	
GRANT	Yes	Yes	

3.2.10. CREATE STATISTICS statement

It is now possible to create extended statistics by specifying a virtual column. [[f7f4052](#)]

Example 54 Creating Extended Statistics for Virtual Generated Columns

```
postgres=> CREATE TABLE data1(col1 INT, col2 INT GENERATED ALWAYS AS
           (col1 + 100) VIRTUAL) ;

CREATE TABLE
postgres=> CREATE STATISTICS data1_stat1 ON (col2) FROM data1 ;

CREATE STATISTICS
```

3.2.11. DELETE / UPDATE statement

The FOR PORTION OF clause can now be specified in UPDATE and DELETE statements. When a range is specified in the FOR PORTION OF clause, a new tuple matching that range is created, and the updates specified in the SET clause are applied to it. The original tuple is split into tuples that fall outside the range specified in the FOR PORTION OF clause. [[8e72d91](#)]

Syntax 12 FOR PORTION OF clause

```
UPDATE table_name FOR PORTION OF column FROM start TO end SET ... WHERE ...  
DELETE FROM table_name FOR PORTION OF column FROM start TO end WHERE ...
```

Since the behavior can be confusing, a sample will be used to explain it. Initially, the `range1` table contains two tuples: one in the `col1` column (of type `DATERANGE`) representing the range "April 1, 2026 – May 1, 2026" and another representing the range "May 1, 2026 – June 1, 2026".

Example 55 Tuples before the update

```
postgres=> SELECT * FROM range1 ORDER BY col1 ;  
          col1          | col2  
-----+-----  
 [2026-04-01, 2026-05-01) | 100  
 [2026-05-01, 2026-06-01) | 200  
(2 rows)
```

If the FOR PORTION OF clause is set to "2026/04/10–2026/04/20", the tuples to be updated are selected (tuples where `col2=100`). These tuples are split into the following three tuples.

- 2026/04/01-2026/04/10 (The first half of the original tuple range)
- 2026/04/10-2026/04/20 (Tuple to be updated; updated using a SET clause)
- 2026/04/20-2026/05/01 (The second half of the original tuple range)

Example 56 Tuples after update

```
postgres=> UPDATE range1 FOR PORTION OF col1 FROM '2026-04-10' TO '2026-04-20'
           SET col2=300 ;
```

UPDATE 1

```
postgres=> SELECT * FROM range1 ORDER BY col1 ;
```

col1	col2
[2026-04-01, 2026-04-10)	100
[2026-04-10, 2026-04-20)	300
[2026-04-20, 2026-05-01)	100
[2026-05-01, 2026-06-01)	200

(4 rows)

3.2.12. EXPLAIN statement

The following features have been added to the EXPLAIN statement.

□ Output of I/O Statistics

The IO option has been added to the EXPLAIN statement to output I/O statistics. The IO option must be specified together with the ANALYZE option. [[681daed](#), [3b1117d](#), [e157fe6](#)]

Example 57 Executing an EXPLAIN (IO) statement

```
postgres=> EXPLAIN (ANALYZE, IO, COSTS OFF) SELECT COUNT(*) FROM data1 ;
           QUERY PLAN
```

Aggregate (actual time=425.023..425.024 rows=1.00 loops=1)

Buffers: shared hit=1099 read=52956

-> Seq Scan on data1 (actual time=0.416..235.808 rows=10000000.00 loops=1)

Prefetch: avg=70.84 max=91 capacity=94

I/O: count=3314 waits=7 size=15.98 in-progress=4.54

Buffers: shared hit=1099 read=52956

Planning Time: 0.059 ms

Execution Time: 425.056 ms

(8 rows)

□ Output of Memoize Estimation Information

Estimation information (Estimates:) is now output when Memoize node is included in the execution plan. [\[4bc62b8\]](#)

Example 58 Outputting Memoize Estimate Information

```
postgres=> EXPLAIN SELECT * FROM data1 AS d1 INNER JOIN data1 AS d2
           ON d1.col1 = d2.col2::numeric AND d1.col2::numeric = d2.col1 ;
           QUERY PLAN

-----
Nested Loop  (cost=0.15..10.49 rows=1 width=36)
-> Seq Scan on data1 d1  (cost=0.00..1.40 rows=40 width=18)
-> Memoize  (cost=0.15..0.41 rows=1 width=18)
    Cache Key: d1.col1, (d1.col2)::numeric
    Cache Mode: logical
    Estimates: capacity=20 distinct keys=20 lookups=40 hit percent=50.00%
-> Index Only Scan using idx1_data1 on data1 d2  (cost=0.14..0.40...
    Index Cond: (col1 = (d1.col2)::numeric)
    Filter: (d1.col1 = (col2)::numeric)

(9 rows)
```

□ Output of WAL FPI Byte Count

The WAL data now includes FPI bytes. [\[5ab0b6a\]](#)

Example 59 Output of FPI byte information

```
postgres=> EXPLAIN (ANALYZE, WAL) UPDATE data1 SET col2='update'
           WHERE col1 < 100 ;

               QUERY PLAN

-----
Update on data1  (cost=0.00..1.50 rows=0 width=0) (actual time=0.225..0.225...
  Buffers: shared hit=121 read=1 dirtied=3
  WAL: records=80 fpi=2 bytes=9790 fpi bytes=3656
    -> Seq Scan on data1  (cost=0.00..1.50 rows=40 width=38) (actual time=0. ...
      Filter: (col1 < '100'::numeric)
      Buffers: shared hit=1
Planning Time: 0.281 ms
Execution Time: 0.363 ms
(8 rows)
```

□ Subquery Name

The subquery blocks in the executed execution plan now display unique names. Previously, only numbers were displayed. [[8c49a48](#)]

Example 60 PostgreSQL 19 Output

```
postgres=> EXPLAIN (COSTS OFF) SELECT COUNT(*) FROM data1 WHERE col1 =  
           (SELECT MAX(col1) FROM data1) ;  
           QUERY PLAN  
  
-----  
Aggregate  
  InitPlan expr_1  
    -> Result  
      Replaces: MinMaxAggregate  
      InitPlan minmax_1  
        -> Limit  
          -> Index Only Scan Backward using data1_pkey on data1  
data1_1  
  -> Index Only Scan using data1_pkey on data1  
      Index Cond: (col1 = (InitPlan expr_1).col1)  
(9 rows)
```

Example 61 PostgreSQL 18 Output

```
postgres=> EXPLAIN (COSTS OFF) SELECT COUNT(*) FROM data1 WHERE col1 =  
           (SELECT MAX(col1) FROM data1) ;  
           QUERY PLAN  
  
-----  
Aggregate  
  InitPlan 2  
    -> Result  
      InitPlan 1  
        -> Limit  
          -> Index Only Scan Backward using data1_pkey on data1  
data1_1  
  -> Index Only Scan using data1_pkey on data1  
      Index Cond: (col1 = (InitPlan 2).col1)  
(8 rows)
```

3.2.13. GRANT statement

It is now possible to specify a grantee other than the current user in the GRANTED BY clause of GRANT and REVOKE statements. In previous versions, this resulted in the error "ERROR: grantor must be current user.". [\[dd1398f\]](#)

Example 62 Specifying the GRANTED BY clause

```
postgres=> CREATE TABLE data1(col1 INT PRIMARY KEY, col2 VARCHAR(10)) ;
CREATE TABLE
postgres=> GRANT SELECT ON data1 TO demo2 WITH GRANT OPTION ;
GRANT
postgres=> \connect postgres postgres
You are now connected to database "postgres" as user "postgres".
postgres=# GRANT SELECT ON data1 TO demo3 GRANTED BY demo2 ;
GRANT
postgres=# \dp+
```

Access privileges					
Schema	Name	Type	Access privileges	Column privileges	Policies
public	data1	table	demo=arwdDxtm/demo+		
			demo2=r*/demo	+	
			demo3=r/demo2		

(1 row)

3.2.14. INSERT statement

It is now possible to specify the DO SELECT clause in an INSERT ON CONFLICT statement. [\[8832709\]](#)

Syntax 13 ON CONFLICT DO SELECT

```
INSERT INTO table_name VALUES (...) ON CONFLICT (column_name)
DO SELECT [FOR {UPDATE}|{SHARE}] [RETURNING ...]
```

Example 63 INSERT ON CONFLICT DO SELECT statement

```
postgres=> INSERT INTO data1 VALUES (100, 'conflict100') ON CONFLICT (col1)
            DO SELECT FOR UPDATE RETURNING * ;

 col1 |   col2
-----+-----
   100 | conflict100
(1 row)

INSERT 0 1
```

3.2.15. REPACK statement

The REPACK statement has been added to PostgreSQL 19. The REPACK statement is the SQL statement that executes both the VACUUM FULL and CLUSTER statements in the single command. When the REPACK statement is executed on its own, it targets all tables and materialized views for which the current user has MAINTAIN privileges. It is also possible to specify specific tables, but UNLOGGED tables and partitioned tables cannot be specified. The USING INDEX clause sorts the data based on the information in the specified index. The progress of the REPACK statement can be monitored via the pg_stat_progress_repack view. [[ac58465](#), [28d534e](#), [e76d8c7](#), [5dbb63f](#)]

Syntax 14 REPACK statement

```
REPACK [(options)] [object] [USING INDEX [index_name]]
```

The following options can be specified in a REPACK statement.

Table 24 Options for the REPACK statement

Option	Description
ANALYZE	Execute an ANALYZE statement after the REPACK operation is complete
CONCURRENTLY	Minimize locks and allow access to other SQL statements
VERBOSE	Output a detailed execution log

Example 64 Executing the REPACK statement

```
postgres=> REPACK (ANALYZE, VERBOSE) data1 ;
INFO: repacking "public.data1" in physical order
INFO: "public.data1": found 1666667 removable, 3333333 nonremovable row
versions in 27028 pages
DETAIL: 0 dead row versions cannot be removed yet.
CPU: user: 0.58 s, system: 0.20 s, elapsed: 0.94 s.
INFO: analyzing "public.data1"
INFO: "data1": scanned 18019 of 18019 pages, containing 3333333 live rows and
0 dead rows; 30000 rows in sample, 3333333 estimated total rows
INFO: finished analyzing table "postgres.public.data1"
avg read rate: 1582.075 MB/s, avg write rate: 0.088 MB/s
buffer usage: 23 hits, 18023 reads, 1 dirtied
WAL usage: 3 records, 1 full page images, 2165 bytes, 1560 full page image
bytes, 0 buffers full
system usage: CPU: user: 0.06 s, system: 0.01 s, elapsed: 0.08 s
REPACK
```

If the `CONCURRENTLY` option is specified in the `REPACK` statement, the `SHARE UPDATE EXCLUSIVE` lock is used on the table. Update transactions during the `REPACK` statement execution generate update changes via the logical decoding feature. These update changes are ultimately applied to the new table. During the execution of a `REPACK` statement, a temporary logical replication slot named `"repack_{ID}"` is automatically generated internally. The number of replication slots reserved for `REPACK` is determined by the `max_repack_replication_slots` parameter.

3.2.16. SELECT statement

The following features have been added to `SELECT` statements.

□ GROUP BY ALL clause

The `ALL` clause can now be specified in the `GROUP BY` clause. This executes the `GROUP BY` operation using all output columns. [[cf38a4d](#)]

Example 65 Specifying the GROUP BY ALL clause

```
postgres=> SELECT col1, col2 FROM data1 GROUP BY ALL ORDER BY 1 ;
 col1 | col2
-----+-----
      1 | data1
      2 | data2
      3 | data3
      4 | data4
(4 rows)
```

□ GROUPING SETS and Subqueries

Subqueries now support references to GROUPING SETS columns. The following syntax would have resulted in an error in previous versions. [\[415100a\]](#)

Example 66 GROUPING SETS and Subqueries

```
postgres=> SELECT (SELECT col1),
                  (SELECT col2),
                  (SELECT GROUPING(col1, col2)),
                  (SELECT SUM(t1.col3)),
                  (SELECT max(t2.col3))
FROM
    gctest1 t1 FULL JOIN gctest2 t2 USING (col1, col2)
GROUP BY GROUPING SETS ((col1, col2), ());
 col1 | col2 | grouping | sum | max
-----+-----+-----+----+----
      |      |          |    |    |
    1 |    1 |          |  0 | 147 |  2
    1 |    3 |          |  0 |  14 |
    3 |    4 |          |  0 |  17 |
    3 |    3 |          |  0 |  16 |
    1 |    2 |          |  0 |  25 |  2
    2 |    2 |          |  0 |     |  2
    4 |    1 |          |  0 |  37 |
    2 |    3 |          |  0 |  15 |
(9 rows)
```

3.2.17. SET statement

It is now possible to specify a NULL value in SET statements and ALTER SYSTEM SET statements.

[\[ff4597a\]](#)

Example 67 Specifying NULL values in the SET statement

```
postgres=> SHOW search_path ;
      search_path
-----
"$user", public
(1 row)

postgres=> SET search_path TO NULL ;
SET
postgres=> SHOW search_path ;
      search_path
-----

(1 row)
```

3.2.18. VACUUM statement

The following information has been added to the output of the VACUUM VERBOSE statement.

□ Memory Information

Memory information beginning with "memory usage:" is output. [\[736f754e\]](#)

Example 68 Memory Information

```
postgres=> VACUUM (VERBOSE) data1 ;
INFO:  vacuuming "postgres.public.data1"
...
WAL usage: 185293 records, 92311 full page images, 613464473 bytes, 576198816
full page image bytes, 1416
memory usage: dead item storage 2.58 MB accumulated across 1 reset (limit 64.00
MB each)
system usage: CPU: user: 0.33 s, system: 0.95 s, elapsed: 32.05 s
VACUUM
```

□ Parallel Worker Information

Information about the parallel workers used by the VACUUM statement is output. [[adcdbe9](#)]

Example 69 Parallel Worker Information

```
postgres=> VACUUM (PARALLEL 2, VERBOSE) data1 ;
INFO:  vacuuming "postgres.public.data1"
INFO:  launched 1 parallel vacuum worker for index vacuuming (planned: 2)
INFO:  finished vacuuming "postgres.public.data1": index scans: 1
...
item identifiers removed
parallel workers: index vacuum: 1 planned, 1 launched in total
index "data1_pkey": pages: 27421 in total, 0 newly deleted, 0 currently deleted,
0 reusable
...
system usage: CPU: user: 0.11 s, system: 1.35 s, elapsed: 5.66 s
VACUUM
```

3.2.19. WAIT FOR LSN statement

The WAIT FOR LSN statement, a new SQL syntax, has been added to PostgreSQL 19. This SQL statement is mainly executed on standby instances in a streaming replication environment. When the WAIT FOR LSN statement is executed, it waits until the specified LSN is applied. [[447aae1](#), [3b4e53a](#), [49a181b](#)]

Syntax 15 WAIT FOR LSN statement

```
WAIT FOR LSN ' /sn' [WITH (option [, ...])]
```

The following options are available.

□ MODE 'mode'

Specify the mode for how long to wait. The following values can be specified. "primary_flush" is specified on the primary server; all others are specified on the standby server.

Table 25 MODE Option Settings

Setting value	Description	Note
primary_flush	Primary server flush	On the primary server
standby_replay	Apply WAL to the standby server	
standby_write	Write WAL to the standby server	
standby_flush	Flush WAL on the standby server	

□ TIMEOUT 'timeout'

Specify the maximum wait time. If no value is specified or if zero (0) is specified, this statement will wait indefinitely.

□ NO_THROW

Even if a timeout occurs, no error is thrown.

Example 70 Executing the WAIT FOR LSN statement

```
postgres=> WAIT FOR LSN '0/0301EF70' ;
      status
-----
      success
(1 row)

postgres=> WAIT FOR LSN '0/0301FF00' WITH (TIMEOUT '10s') ;
ERROR:  timed out while waiting for target LSN 0/0301FF00 to be replayed:
current standby_replay LSN 0/0301EF70
postgres=> WAIT FOR LSN '0/0301FF00' WITH (TIMEOUT '10s', NO_THROW) ;
      status
-----
      timeout
(1 row)
```

The WAIT FOR LSN statement returns the following value.

Table 26 Return value of the WAIT FOR LSN statement

Return value	Description	Note
success	The specified LSN has been reached	
timeout	The wait time has exceeded the timeout	
not in recovery	The executed instance is not in recovery state	

3.2.20. Functions

The following functions have been added or enhanced.

□ Window Functions

It is now possible to specify the IGNORE NULLS clause (ignore NULL values) and the RESPECT NULLS clause (include NULL values) for the window functions LEAD, LAG, FIRST_VALUE, LAST_VALUE, and NTH_VALUE. The default is RESPECT NULLS. [[25a30bb](#)]

Example 71 Specifying the IGNORE NULLS clause and the RESPECT NULLS clause

```
postgres=> SELECT LEAD(col1) IGNORE NULLS OVER (), LAG(col1) RESPECT NULLS OVER  
() FROM data1 ;
```

lead	lag
2	
3	1
4	2
5	3
6	4
...	

□ Function encode and decode

It is now possible to specify base64url and base32hex as encoding formats. [[e1d9171](#), [497c117](#)]

Example 72 Base64 URL encoding and Base32 hex encoding

```
postgres=> SELECT encode(' \x69b73eff', 'base64url'),  
                decode('abc_w', 'base64url') ;
```

encode	decode
abc_w	\x69b73eff

(1 row)

```
postgres=> SELECT encode(' \x1122334455', 'base32hex'),  
                decode('24H36H2L', 'base32hex') ;
```

encode	decode
24H36H2L	\x1122334455

(1 row)

□ Function error_on_null

The error_on_null function has been added; it returns an error if the input value is NULL, and returns the input value if it is not NULL. [[2b75c38](#)]

Syntax 16 Function error_on_null

```
anyelement error_on_null (anyelement)
```

Example 73 Execution of the error_on_null function

```
postgres=> SELECT error_on_null (NULL::INT) ;  
ERROR: null value not allowed
```

□ Function random

Variations of the random function that specify maximum and minimum values now support the date, timestamp, and timestamp with time zone types. [[faf071b](#)]

Syntax 17 Function random variations

```
date random(min date, max date)  
timestamp random(min timestamp, max timestamp)  
timestampz random(min timestampz, max timestampz)
```

Example 74 Variations of the random function

```
postgres=> SELECT random('2024-09-09'::date, '2025-09-09'::date),  
                random('2024-09-09'::timestampz, '2025-09-09'::timestampz) ;  
 random   | random  
-----+-----  
 2025-02-04 | 2025-02-16 07:24:48.518771+09  
(1 row)
```

□ Function range_minus_multi and multirange_minus_multi

The range_minus_multi and multirange_minus_multi functions have been added to remove ranges from a range and return multiple ranges, respectively. [[5eed8ce](#)]

Syntax 18 Function range_minus_multi and multirange_minus_multi

```
SETOF anyrange range_minus_multi(anyrange, anyrange)  
SETOF anymultirange multirange_minus_multi(anymultirange, anymultirange)
```

Example 75 The range_minus_multi function and the multirange_minus_multi function

```
postgres=> SELECT range_minus_multi(numrange(1.0, 3.0), numrange(1.5, 2.0)) ;
range_minus_multi
-----
[1.0, 1.5)
[2.0, 3.0)
(2 rows)

postgres=> SELECT multirange_minus_multi(nummultirange(numrange(1, 4)),
multirange_minus_multi
-----
{[1, 2), [3, 4)}
(1 row)
```

□ Function tid_block and tid_offset

The tid_block and tid_offset functions have been added to retrieve the block number and row offset from the CTID. [[df6949c](#)]

Syntax 19 Function tid_block and tid_offset

```
bigint tid_block(tid)
integer tid_offset(tid)
```

Example 76 Executing the tid_block and tid_offset functions

```
postgres=> SELECT ctid, tid_block(ctid), tid_offset(ctid), col1 FROM data1 ;
ctid | tid_block | tid_offset | col1
-----+-----+-----+-----
(0,1) |          0 |          1 | 100
(0,2) |          0 |          2 | 200
(2 rows)
```

□ Function width_bucket

It is now possible to specify NaN as the first parameter, operand, of the width_bucket function. The following SQL statement would have caused an error in previous versions. [\[7374b3a\]](#)

Example 77 Executing the width_bucket function

```
postgres=> SELECT width_bucket(' NaN', 3.0, 4.0, 888) ;
width_bucket
-----
            889
(1 row)
```

□ DDL Export

Functions have been added to output DDL for databases, tablespaces, and roles. [\[4881981\]](#), [76e514e](#), [b99fd9f](#), [a4f774c](#)

Syntax 20 Export DDL functions

```
SETOF text pg_get_database_ddl(regdatabase, VARIADIC text DEFAULT NULL::text)
SETOF text pg_get_tablespace_ddl(name, VARIADIC text DEFAULT NULL::text)
SETOF text pg_get_role_ddl(regrole, VARIADIC text DEFAULT NULL::text)
```

The following settings can be specified in the options.

Table 27 Available options

Option	Data type	Description	Note
pretty	boolean	Formatted output	
owner	boolean	Include owner	Cannot be specified with pg_get_role_ddl
tablespace	boolean	Include tablespace	pg_get_database_ddl only
memberships	boolean	Include member	pg_get_role_ddl only

Example 78 Retrieving DDL for a database

```
postgres=> SELECT pg_get_database_ddl('postgres', 'pretty', 'true') ;
           pg_get_database_ddl
-----
CREATE DATABASE postgres          +
    WITH TEMPLATE = template0     +
    ENCODING = 'UTF8'             +
    LOCALE_PROVIDER = libc        +
    LOCALE = 'C' ;
ALTER DATABASE postgres OWNER TO postgres;
(2 rows)
```

□ Function `pg_clear_extended_stats`

The `pg_clear_extended_stats` function, which clears a table's extended statistics, has been added. [\[d756fa1\]](#)

Syntax 21 Function `pg_clear_extended_stats`

```
void pg_clear_extended_stats(schemaname text, relname text,
                             statistics_schemaname text, statistics_name text, inherited boolean)
```

Example 79 Executing of the `pg_clear_extended_stats`

```
postgres=> CREATE STATISTICS data1_stat1 ON col1,col2 FROM data1 ;
CREATE STATISTICS
postgres=> SELECT pg_clear_extended_stats('public', 'data1', 'public',
                                           'data1_stat1', true) ;
-[ RECORD 1 ]-----+
pg_clear_extended_stats |
```

□ Function `pg_get_multixact_stats`

The `pg_get_multixact_stats` function, which retrieves the status of multi-transactions, has been added. This function returns the following values. [\[97b1017\]](#)

Table 28 Return value of the pg_get_multixact_stats function

Column name	Data type	Description
num_mxids	bigint	Multi-transaction IDs in use
num_members	bigint	Size of multi-transaction ID members in use
members_size	bigint	Storage size used by members
oldest_multixact	xid	Oldest multi-transaction ID

Example 80 Execution of the pg_get_multixact_stats function

```
postgres=# SELECT * FROM pg_get_multixact_stats() ;
 num_mxids | num_members | members_size | oldest_multixact
-----+-----+-----+-----
          1 |           2 |           0 |              1
(1 row)
```

□ Function pg_get_sequence_data

The page_lsn column, which indicates the LSN, has been added to the output results. [\[b93172c\]](#)

Example 81 Executing of the pg_get_sequence_data function

```
postgres=> SELECT * FROM pg_get_sequence_data(' seq01' ) ;
 last_value | is_called | page_lsn
-----+-----+-----
          4 | t         | 3/0701C0A0
(1 row)
```

□ Function pg_get_publication_tables

The pg_get_publication_tables function now includes a variant that allows table information to be specified. [\[fd7a25a\]](#)

Syntax 22 Function pg_get_publication_tables with table id

```
SETOF record pg_get_publication_tables(pubnames text[], target_relid oid)
```

Example 82 Executing of the pg_get_publication_tables function

```
postgres=> SELECT * FROM
            pg_get_publication_tables(' {pub1}', 'data1'::regclass::oid) ;
 pubid | relid | attrs | qual
-----+-----+-----+-----
 16405 | 16399 | 1 2   |
(1 row)
```

□ Function pg_replication_origin_session_setup

The pid parameter has been added to specify the worker process ID using origin. The default value is 0, which indicates the first process ID. [[5b14870](#)]

[[5b14870](#)]

Example 83 Definition of the pg_replication_origin_session_setup function

```
postgres=> \dfS pg_replication_origin_session_setup
List of functions
-[ RECORD 1 ]-----+-----
Schema          | pg_catalog
Name             | pg_replication_origin_session_setup
Result data type | void
Argument data types | node_name text, pid integer DEFAULT 0
Type            | func
```

□ Function pg_restore_extended_stats

The pg_restore_extended_stats function, which restores a table's extended statistics, has been added. [[0e80f3f](#), [302879b](#), [ba97bf9](#)]

Syntax 23 Function pg_restore_extended_stats

```
boolean pg_restore_extended_stats(VARIADIC kwargs "any")
```

Example 84 Executing of the pg_restore_extended_stats function

```
postgres=> CREATE STATISTICS stat1_data1 ON col1,col2 FROM data1 ;
CREATE STATISTICS
postgres=> SELECT pg_restore_extended_stats(
    'schemaname', 'public',
    'relname', 'data1',
    'statistics_schemaname', 'public',
    'statistics_name', 'stat1_data1',
    'inherited', false,
    'n_distinct', ' [{"attributes" : [1,2], "ndistinct" : 4}]'::pg_ndistinct) ;
-[ RECORD 1 ]-----+--
pg_restore_extended_stats | t
```

□ Function pg_stat_get_backend_wal

The `pg_stat_get_backend_wal` function now includes `wal_fpi_bytes` (of type numeric), which indicates the number of FPI bytes, in its return value. [[f9a09aa](#)]

□ Function pg_sync_replication_slots

The `pg_sync_replication_slots` function now includes a feature that retries until synchronization is complete. [[0d2d4a0](#)]

3.2.21. JSON

The following enhancements regarding JSON data have been implemented.

□ Added jsonpath methods

The following jsonpath methods have been added. [[bd4f879](#)]

Table 29 Added string methods

Method name	Description	Note
ltrim / rtrim / btrim	Remove leading and trailing characters	
lower	Convert to lowercase	
upper	Convert to uppercase	
initcap	Capitalize the first letter and convert the rest to lowercase	
replace	Convert the specified string	
split_part	Return elements split by the specified string	

Example 85 Executing the newly added jsonpath string method

```

postgres=> SELECT jsonb_path_query('"xTRIMx"'::jsonb, '$.btrim("x")') ;
-[ RECORD 1 ]-----+-----
jsonb_path_query | "TRIM"

postgres=> SELECT jsonb_path_query('"LOWER"'::jsonb, '$.lower()') ;
-[ RECORD 1 ]-----+-----
jsonb_path_query | "lower"

postgres=> SELECT jsonb_path_query('"upper"'::jsonb, '$.upper()') ;
-[ RECORD 1 ]-----+-----
jsonb_path_query | "UPPER"

postgres=> SELECT jsonb_path_query('"iNITCaP"'::jsonb, '$.initcap()') ;
-[ RECORD 1 ]-----+-----
jsonb_path_query | "Initcap"

postgres=> SELECT jsonb_path_query('"REpLACE"'::jsonb, '$.replace("p","P")') ;
-[ RECORD 1 ]-----+-----
jsonb_path_query | "REPLACE"

postgres=> SELECT jsonb_path_query('"test@data"'::jsonb,
                                '$.split_part("@",2)') ;
-[ RECORD 1 ]-----+-----
jsonb_path_query | "data"

```

□ IS JSON predicate

It is now possible to specify the IS JSON predicate for the domain. In previous versions, the IS JSON predicate was only available for the text, json, jsonb, and bytea data types. [\[3b4c2b9\]](#)

Example 86 IS JSON Predicate for the Domain

```
postgres=> CREATE DOMAIN jd1 AS jsonb CHECK ((VALUE ->'val')::text <> '3') ;
CREATE DOMAIN
postgres=> SELECT ' {"val": 1} '::jd1 IS JSON ;
?column?
-----
t
(1 row)

postgres=> SELECT ' {"val": 3} '::jd1 IS JSON ;
ERROR:  value for domain jd1 violates check constraint "jd1_check"
```

3.2.22. Data Types

The following extensions have been implemented for data types in PostgreSQL 19.

□ Data type regdatabase

The regdatabase data type, which represents a database OID, has been added. To retrieve the OID of the current database, it can be written as follows. [\[bd09f02\]](#)

Example 87 Querying by regdatabase data type

```
postgres=> SELECT * FROM pg_shdepend WHERE
            dbid = current_database()::regdatabase ;
 dbid | classid | objid | objsubid | refclassid | refobjid | deptype
-----+-----+-----+-----+-----+-----+-----
    5 |    2615 |   2200 |         0 |          1260 |    16384 | a
(1 row)
```

□ Data type oid8

The oid8 data type, which represents 8-byte OIDs, has been added. [\[b139bd3\]](#)

Example 88 Data type oid8

```
postgres=> SELECT 999999999::oid8 ;
      oid8
-----
 999999999
(1 row)
```

□ Converting between the UUID and BYTEA types

The uuid and bytea types can now be converted between each other. [[ba21f5b](#)]

Example 89 Data type conversion between UUID and BYTEA

```
postgres=> SELECT '\x019a2f859ced7225b99d9c55044a2563'::bytea::uuid ;
      uuid
-----
019a2f85-9ced-7225-b99d-9c55044a2563
(1 row)

postgres=> SELECT '5b35380a-7143-4912-9b55-f322699c6770'::uuid::bytea ;
      bytea
-----
\x5b35380a714349129b55f322699c6770
(1 row)
```

□ GiST Indexes of Type inet and cidr

The operator classes for GiST indexes of type inet and cidr have been changed to inet_ops and cidr_ops, respectively. As a result of this change, the btree_gist extension is no longer required. The following CREATE INDEX statements will result in an error in PostgreSQL 18 and earlier. The pg_upgrade command will fail if the older database contains the gist_inet_ops or gist_cidr_ops operator classes. [[b352d3d](#)]

Example 90 GiST indexes for inet and cidr types

```
postgres=> CREATE TABLE data1(col1 INET, col2 CIDR) ;
CREATE TABLE
postgres=> CREATE INDEX idx1_data1 ON data1 USING gist (col1) ;
CREATE INDEX
postgres=> CREATE INDEX idx2_data1 ON data1 USING gist (col2) ;
CREATE INDEX
```

□ Tsearch-related data types

The analysis now uses the database's default collation. Previous versions used the database's CTYPE setting. [[e113f9c](#)]

3.2.23. PL/pgSQL

The following features have been added to the PL/pgSQL module.

□ Optimizing Simple SELECT INTO Statements

The execution of the SELECT INTO statement that uses a simple scalar expression to assign to a single variable is no longer treated as an SQL statement execution; instead, it is now converted into an assignment operation. Due to this update, the target SQL statement will no longer trigger hooks in the pg_stat_statements module. [[ce8d5fe](#)]

□ Reserved Word Removal

EXECUTE and STRICT have been removed from the reserved word list. These words can now be used as variable names and for other purposes. [[999f172](#)]

Example 91 Using reserved keywords as variable names

```
postgres=> DO $$
DECLARE
EXECUTE INT;
BEGIN
EXECUTE := 100;
END;
$$;
DO
postgres=>
```

3.2.24. PL/Python

Events trigger support has been added. [[53eff47](#)]

3.2.25. Optimizer

The faster execution plan is now selected.

□ Eager Aggregation

Eager aggregation optimization has been implemented to reduce the number of input tuples in join operations by performing partial aggregation before the join. This optimization is controlled by the `enable_eager_aggregate` parameter. The default value is "on", meaning it is enabled. The group size required to consider eager aggregation is specified by the `min_eager_agg_group_size` parameter. The default value for this parameter is 8. [[8e11859](#)]

Example 92 PostgreSQL 18 Execution Plan

```
postgres=> EXPLAIN (VERBOSE, COSTS OFF) SELECT t1.col1, avg(t2.col3)
           FROM eager1 AS t1 JOIN eager2 AS t2 ON t1.col2 = t2.col2
           GROUP BY t1.col1 ORDER BY t1.col1 ;
           QUERY PLAN
```

GroupAggregate

Output: t1.col1, avg(t2.col3)

Group Key: t1.col1

-> Sort

Output: t1.col1, t2.col3

Sort Key: t1.col1

-> Hash Join

Output: t1.col1, t2.col3

Hash Cond: (t2.col2 = t1.col2)

-> Seq Scan on public.eager2 t2

Output: t2.col1, t2.col2, t2.col3

-> Hash

Output: t1.col1, t1.col2

-> Seq Scan on public.eager1 t1

Output: t1.col1, t1.col2

(15 rows)

Example 93 PostgreSQL 19 Execution Plan

```
postgres=> EXPLAIN (VERBOSE, COSTS OFF) SELECT t1.col1, avg(t2.col3)
           FROM eager1 AS t1 JOIN eager2 AS t2 ON t1.col2 = t2.col2
           GROUP BY t1.col1 ORDER BY t1.col1 ;
           QUERY PLAN
```

Finalize GroupAggregate

Output: t1.col1, avg(t2.col3)

Group Key: t1.col1

-> Sort

Output: t1.col1, (PARTIAL avg(t2.col3))

Sort Key: t1.col1

-> Hash Join

Output: t1.col1, (PARTIAL avg(t2.col3))

Hash Cond: (t1.col2 = t2.col2)

-> Seq Scan on public.eager1 t1

Output: t1.col1, t1.col2, t1.col3

-> Hash

Output: t2.col2, (PARTIAL avg(t2.col3))

-> Partial HashAggregate

Output: t2.col2, PARTIAL avg(t2.col3)

Group Key: t2.col2

-> Seq Scan on public.eager2 t2

Output: t2.col1, t2.col2, t2.col3

(18 rows)

□ Using Memoize with ANTI JOIN

The memoization feature is now available for SEMI JOIN and ANTI JOIN. [[0da29e4](#)]

Example 94 PostgreSQL 18 Execution Plan

```
postgres=> EXPLAIN (COSTS OFF, BUFFERS OFF)
SELECT COUNT(*) FROM data1 d1 LEFT JOIN
LATERAL (SELECT DISTINCT ON (c1) c1, c2, d1.c1 AS x FROM data1 d2) d2
ON d1.c1 + 1 = d2.c1 WHERE d2.c1 IS NULL ;
```

QUERY PLAN

Aggregate

- > Nested Loop Anti Join
 - > Seq Scan on data1 d1
 - > Subquery Scan on d2
 - Filter: ((d1.c1 + 1) = d2.c1)
 - > Unique
 - > Sort
 - Sort Key: d2_1.c1
 - > Seq Scan on data1 d2_1

(9 rows)

Example 95 PostgreSQL 19 Execution Plan

```
postgres=> EXPLAIN (COSTS OFF, BUFFERS OFF)
SELECT COUNT(*) FROM data1 d1 LEFT JOIN
LATERAL (SELECT DISTINCT ON (c1) c1, c2, d1.c1 AS x FROM data1 d2) d2
ON d1.c1 + 1 = d2.c1 WHERE d2.c1 IS NULL ;
          QUERY PLAN
```

Aggregate

- > Nested Loop Anti Join
 - > Seq Scan on data1 d1
 - > Memoize
 - Cache Key: (d1.c1 + 1), d1.c1
 - Cache Mode: binary
 - > Subquery Scan on d2
 - Filter: ((d1.c1 + 1) = d2.c1)
 - > Unique
 - > Sort
 - Sort Key: d2_1.c1

(12 rows)

□ Rewriting the COUNT function

If possible, the COUNT(any) clause is rewritten as a COUNT(*) clause. This rewrite is limited to cases where a non-NULL column is specified and there is no ORDER BY or DISTINCT clause. This change reduces overhead and improves performance. [\[42473b3\]](#)

Example 96 PostgreSQL 18 Execution Plan

```
postgres=> EXPLAIN (VERBOSE) SELECT COUNT(col1) FROM data1 ;
               QUERY PLAN
-----
Aggregate  (cost=17906.00..17906.01 rows=1 width=8)
  Output: count(col1)
    -> Seq Scan on public.data1  (cost=0.00..15406.00 rows=1000000 width=4)
      Output: col1, col2
(4 rows)
```

Example 97 PostgreSQL 19 Execution Plan

```
postgres=> EXPLAIN (VERBOSE) SELECT COUNT(col1) FROM data1 ;
               QUERY PLAN
-----
Aggregate  (cost=17906.00..17906.01 rows=1 width=8)
  Output: count(*)
    -> Seq Scan on public.data1  (cost=0.00..15406.00 rows=1000000 width=0)
      Output: col1, col2
(4 rows)
```

□ Parallelization of TID Range Scans

It is now possible to execute TID Range Scans using parallel queries. [\[0ca3b16\]](#)

Example 98 PostgreSQL 18 Execution Plan

```
postgres=> EXPLAIN SELECT * FROM data1 WHERE ctid > ' (10,0)' AND ctid < ' (30,1)';
               QUERY PLAN
-----
Tid Range Scan on data1  (cost=0.01..61.00 rows=3700 width=10)
  TID Cond: ((ctid > ' (10,0)'::tid) AND (ctid < ' (30,1)'::tid))
(2 rows)
```

Example 99 PostgreSQL 19 Execution Plan

```
postgres=> EXPLAIN SELECT * FROM data1 WHERE ctid > ' (10,0)' AND ctid < ' (30,1)';
               QUERY PLAN
-----
Gather  (cost=0.01..33.25 rows=3700 width=10)
  Workers Planned: 4
    -> Parallel Tid Range Scan on data1  (cost=0.01..33.25 rows=925 width=10)
          TID Cond: ((ctid > ' (10,0)::tid) AND (ctid < ' (30,1)::tid))
(4 rows)
```

□ Partial Hash Index

HASH indexes created as partial indexes can now be used in execution plans when a WHERE clause is specified that matches their conditions. This feature has been backported to PostgreSQL 14 and later. [\[5528e8d\]](#)

Example 100 PostgreSQL 18 Execution Plan

```
postgres=> CREATE INDEX idx1_data1 ON data1 USING hash (col2) WHERE col2=1000 ;
CREATE INDEX
postgres=> EXPLAIN SELECT * FROM data1 WHERE col2=2000 ;
               QUERY PLAN
-----
Seq Scan on data1  (cost=0.00..17906.00 rows=1 width=14)
  Filter: (col2 = 2000)
(2 rows)
```

Example 101 PostgreSQL 19 Execution Plan

```
postgres=> CREATE INDEX idx1_data1 ON data1 USING hash (col2) WHERE col2=1000 ;
CREATE INDEX
postgres=> EXPLAIN SELECT * FROM data1 WHERE col2=1000 ;
               QUERY PLAN
-----
Index Scan using idx1_data1 on data1  (cost=0.00..8.02 rows=1 width=14)
  Index Cond: (col2 = 1000)
(2 rows)
```

□ Optimization of the IS DISTINCT FROM clause

The IS DISTINCT FROM clause for columns with a NOT NULL constraint is now converted to the "<>" operator. [[0a37961](#)]

Example 102 PostgreSQL 18 Execution Plan

```
postgres=> EXPLAIN SELECT * FROM data1 WHERE col1 IS DISTINCT FROM col1 ;
               QUERY PLAN
-----
Seq Scan on data1  (cost=0.00..17906.00 rows=995000 width=10)
  Filter: (col1 IS DISTINCT FROM col1)
(2 rows)
```

Example 103 PostgreSQL 19 Execution Plan

```
postgres=> EXPLAIN SELECT * FROM data1 WHERE col1 IS DISTINCT FROM col1 ;
               QUERY PLAN
-----
Seq Scan on data1  (cost=0.00..17906.00 rows=995000 width=10)
  Filter: (col1 <> col1)
(2 rows)
```

□ Anti-Join Transformation of the NOT IN Clause

If it is determined to be safe, the NOT IN clause uses an Anti-Join in the execution plan. [[383eb21](#)]

Example 104 PostgreSQL 18 Execution Plan

```
postgres=> EXPLAIN SELECT * FROM data1 WHERE col1 NOT IN (SELECT col1 FROM
              data1 WHERE col1 < 10000) ;
              QUERY PLAN

-----
Seq Scan on data1 (cost=194.99..364.97 rows=5000 width=8)
  Filter: (NOT (ANY (col1 = (hashed SubPlan 1).col1)))
  SubPlan 1
    -> Seq Scan on data1 data1_1 (cost=0.00..169.99 rows=9999 width=4)
        Filter: (col1 < 10000)
(5 rows)
```

Example 105 PostgreSQL 19 Execution Plan

```
postgres=> EXPLAIN SELECT * FROM data1 WHERE col1 NOT IN (SELECT col1 FROM
              data1 WHERE col1 < 10000) ;
              QUERY PLAN

-----
Hash Anti Join (cost=29.98..48.71 rows=1 width=8)
  Hash Cond: (data1.col1 = data1_1.col1)
  -> Seq Scan on data1 (cost=0.00..14.99 rows=999 width=8)
  -> Hash (cost=17.49..17.49 rows=999 width=4)
      -> Seq Scan on data1 data1_1 (cost=0.00..17.49 rows=999 width=4)
          Filter: (col1 < 10000)
(6 rows)
```

□ Details of the Result Node

Information about the source of the result has been added to the Result node. As a result, information from the "Replace:" node will now be included in the execution results of the EXPLAIN statement. [\[f2bae51\]](#)

Example 106 PostgreSQL 19 Execution Plan

```
postgres=> EXPLAIN (COSTS OFF) SELECT * FROM data1 WHERE  
           col1=(SELECT MAX(col1) FROM data1) ;  
           QUERY PLAN
```

Index Scan using data1_pkey on data1

Index Cond: (col1 = (InitPlan expr_1).col1)

InitPlan expr_1

-> Result

Replaces: MinMaxAggregate

InitPlan minmax_1

-> Limit

-> Index Only Scan Backward using data1_pkey on data1 ...

(8 rows)

3.3. Configuration parameters

In PostgreSQL 19, the following parameters have been changed.

3.3.1. Added Parameters

The following parameters have been added. The `file_extend_method` parameter will be backported to PostgreSQL 16 and later. [[06473f5](#), [8e11859](#), [dd3ae37](#), [67c2097](#), [f94e914](#), [1d92e0c](#), [4f43302](#), [d7965d6](#), [a8f45de](#), [1ff3180](#), [e76d8c7](#), [d1c01b7](#)]

Table 30 Added Parameters

Parameter name	Description (Context)	Default value
<code>autovacuum_analyze_score_weight</code>	Specifies the scaling factor of the analyze threshold (sighup)	1.0
<code>autovacuum_freeze_score_weight</code>	Specifies the scaling factor of the transaction ID age (sighup)	1.0
<code>autovacuum_max_parallel_workers</code>	Sets the maximum number of autovacuum parallel workers (sighup)	0
<code>autovacuum_multixact_freeze_score_weight</code>	Specifies the scaling factor of the multixact ID age (sighup)	1.0
<code>autovacuum_vacuum_insert_score_weight</code>	Specifies the scaling factor of the vacuum insert threshold (sighup)	1.0
<code>autovacuum_vacuum_score_weight</code>	Specifies the scaling factor of the vacuum threshold (sighup)	1.0
<code>autovacuum_worker_slots</code>	Specifies the number of backend slots to reserve for autovacuum worker (postmaster)	16
<code>debug_exec_backend</code>	Reports whether PostgreSQL has been built with EXEC_BACKEND enabled (internal)	off
<code>debug_print_raw_parse</code>	Output the parse tree without converting it (user)	off
<code>effective_wal_level</code>	Effective WAL output level (internal)	replica
<code>enable_eager_aggregate</code>	Enable eager aggregation execution plan (user)	on
<code>file_extend_method</code>	File extension methods (sighup)	posix_fallocate

Parameter name	Description (Context)	Default value
hosts_file	SSL/SNI filename (postmaster)	{PGDATA}/pg _hosts.conf
io_max_workers	Maximum number of I/O workers (sighup)	8
io_min_workers	Minimum number of I/O workers (sighup)	2
io_worker_idle_timeout	Maximum idle time for I/O workers (sighup)	60000
io_worker_launch_interval	Minimum interval between starting new I/O workers (sighup)	100ms
log_autoanalyze_min_duration	Log output time for automatic ANALYZE execution (sighup)	10min
max_repack_replication_slots	Number of replication slots reserved for executing REPACK (postmaster)	5
min_eager_agg_group_size	Minimum memory allocation when using eager aggregation (user)	8
password_expiration_warning_threshold	Period for outputting password expiration warnings (sighup)	7d
ssl_sni	Whether to use SNI for SSL connections (sighup)	off
timing_clock_source	Specify the clock source used for timing measurements (superuser)	auto (tsc)
wal_sender_shutdown_timeout	Timeout for stopping the WAL sender process (user)	-1

□ Password Expiration Warning

If the password's expiration date is within the range specified by the `password_expiration_warning_threshold` parameter (default value: 7 days), a warning will be output during authentication. [[1d92e0c](#)]

Example 107 Password Expiration Warning

```
$ psql -h dbvr1 -d postgres -U user1
Password for user user1: <<Password>>
WARNING:  role password will expire soon
DETAIL:   The password for role "user1" will expire in 13 hours.
psql (19beta1)
Type "help" for help.

postgres=>
```

3.3.2. Changed Parameters

The following parameters have been changed in terms of setting range and choices. [[38e0190](#), [1efdd7c](#), [8a6af3a](#), [12da457](#), [bf5da5d](#)]

Table 31 Changed Parameters

Parameter name	Change
data_checksums	If the checksum is currently being enabled, the status changes to "inprogress-on". Data type changed from boolean to enum
extension_control_path	If an extension with the same name exists in the specified path, the first path found is output to the pg_available_extensions view
io_workers	Split into io_max_workers and io_min_workers
log_min_messages	The log output level can now be changed for each process type.
sync_replication_slots	To enable this parameter, the wal_level parameter must be set to logical
wal_receiver_timeout	The context for configuration changes has been changed from signup to user

□ Specifying the log_min_messages parameter

The log_min_messages parameter now supports changing the log output level on a per-process basis. Specify process names and levels separated by colons (:), and separate multiple processes with commas (,). [[38e0190](#)]

Table 32 String for specifying the process

Process name setting	Description	Note
archiver	WAL archiver process	
autovacuum	Autovacuum Launcher, Autovacuum Worker process	
backend	Backend process	
bgworker	Background worker process	
bgwriter	Background writer process	
checkpointer	Checkpointer process	
ioworker	I/O Worker process	
postmaster	Postmaster process	
slotsyncworker	Replication Slot Synchronous Worker process	
startup	Startup process	
syslogger	Logger process	
walreceiver	WAL Receiver process	
walsender	WAL Sender process	
walsummarizer	WAL Summarizer process	
walwriter	WAL Writer process	

Example 108 Changing the log_min_messages parameter

```
postgres=# SET log_min_messages TO 'warning, autovacuum:debug1' ;
SET
```

3.3.3. Changed Default Values

The following parameters have changed default values. [[2aac62b](#), [7f8c88c](#), [79534f9](#)]

Table 33 Parameters whose default values have been changed

Parameter name	PostgreSQL 18	PostgreSQL 19	Note
jit	on	Off	
log_lock_waits	off	On	
max_locks_per_transaction	64	128	
server_version	18.4	19beta1	
server_version_num	180004	190000	

In binaries built using the "configure" command with the --with-lz4 option, the default value for the default_toast_compression parameter is lz4.

3.3.4. Removed Parameters

The following parameters have been removed. [\[4576208\]](#)

Table 34 Removed Parameters

Parameter name	Reason
escape_string_warning	Due to the standard_conforming_strings parameter being fixed, this is no longer necessary.

3.4. Utilities

This section describes the major feature enhancements to the utility commands.

3.4.1. Oid2name

When the `--extended` option is specified, the "Path" field, which indicates the file path, is now output. [\[3c5ec35\]](#)

Example 109 Executing the oid2name command

```
$ oid2name --table=data1 --extended
From database "postgres":
  Filenode  Table Name    Oid  Schema  Tablespace      Path
-----
      16385      data1  16385  public  pg_default  base/5/16385
```

3.4.2. Pg_archivecleanup

The log now includes an entry indicating that dry-run mode has been specified. This feature has also been added to the `pg_createsubscriber` and `pg_combinebackup` commands. [\[c05dee1\]](#)

Example 110 Executing pg_archivecleanup command with dry-run mode

```
$ pg_archivecleanup --dry-run arch 000000010000000000000000A.00000028.backup
pg_archivecleanup: Executing in dry-run mode.
No files will be removed.
arch/00000001000000000000000001
arch/00000001000000000000000002
arch/00000001000000000000000003
...
```

3.4.3. Pg_createsubscriber

The following features have been added to the `pg_createsubscriber` command.

□ Log Output Directory

The `--logdir` option (short form: `-l`) has been added to the `pg_createsubscriber` command to specify the log output directory. [[6b5b7ea](#)]

Example 111 Specifying the log output directory

```
$ pg_createsubscriber -d postgres -P "dbname=postgres user=postgres port=5432"
-l ~/log --publication pub1 --subscription sub1 -D data.s
$ ls ~/log/
20260604T174538.189  20260604T174611.016
$ ls ~/log/20260604T174538.189/pg_createsubscriber_server.log
/home/postgres/log/20260604T174538.189/pg_createsubscriber_server.log
$ cat ~/log/20260604T174538.189/pg_createsubscriber_server.log
LOG:  redirecting log output to logging collector process
HINT:  Future log output will appear in directory "log".
```

□ Reuse of Replication Slots

If a specified replication slot already exists, it will now be reused. [[85ddcc2](#)]

□ Recovery Parameter Configuration File

The recovery parameters that are temporarily set when the `pg_createsubscriber` command is executed are now configured in `pg_createsubscriber.conf`. In previous versions, these were appended to the `postgresql.auto.conf` file. To enable the `pg_createsubscriber.conf` file, `"include_if_exists 'pg_createsubscriber.conf'"` is automatically added to the `postgresql.auto.conf` file. After the recovery process is complete, the filename of the `pg_createsubscriber.conf` file is changed to `pg_createsubscriber.conf.disabled`. [[639352d](#)]

Example 112 Parameter file after executing the pg_createsubscriber command

```
$ pg_createsubscriber -d postgres -p 5433 -P "host=dbsvr1 port=5432
dbname=postgres user=postgres" -D data.s --publication=pub1
--subscription=sub1
LOG: redirecting log output to logging collector process
HINT: Future log output will appear in directory "log".
LOG: redirecting log output to logging collector process
HINT: Future log output will appear in directory "log".
$ tail -1 data.s/postgresql.auto.conf
include_if_exists 'pg_createsubscriber.conf'
$ cat data.s/pg_createsubscriber.conf.disabled
primary_conninfo      = 'user=postgres  passfile='' /home/postgres/.pgpass''
channel_binding=prefer  port=5432  sslmode=prefer  sslnegotiation=postgres
sslcompression=0  sslcertmode=allow  sslsni=1  ssl_min_protocol_version=TLSv1.2
gssencmode=disable  krbsrvname=postgres  gssdelegation=0  target_session_attrs=any
load_balance_hosts=disable'
recovery_target = ''
recovery_target_timeline = 'latest'
recovery_target_inclusive = false
recovery_target_action = promote
recovery_target_name = ''
recovery_target_time = ''
recovery_target_xid = ''
recovery_target_lsn = '0/05000320'
```

3.4.4. Pg_dump

The following features have been added to the pg_dump command.

□ Dump of extended statistics

Extended statistics are now exported using the pg_restore_extended_stats function. [[c32fb29](#)]

Example 113 Dump of extended statistics

```
$ pg_dump -d postgres -U demo --statistics-only | grep restore
SELECT * FROM pg_catalog.pg_restore_relation_stats(
SELECT * FROM pg_catalog.pg_restore_attribute_stats(
SELECT * FROM pg_catalog.pg_restore_attribute_stats(
SELECT * FROM pg_catalog.pg_restore_attribute_stats(
SELECT * FROM pg_catalog.pg_restore_extended_stats(
```

3.4.5. Pg_dumpall

The pg_dumpall command now supports output formats other than text. The output format is specified using the --format option (short form: -F). The default value remains text (--format=p). The following values can be specified for the --format option. For output formats other than text, a directory name must be specified for the output destination (--file option). When restoring data dumped by the pg_dumpall command, use the pg_restore command unless text format was specified. [[763aaa0](#)]

Table 35 Format specification

Format value	Description	Note
p	Text format	Default value
t	Custom dump format (file extension: .dmp)	
c	tar format (file extension: .tar)	
d	gzip format by table (file extension: .gz)	

Example 114 Dump in gzip format

```
$ pg_dumpall -format=d --file=dump.d
$ ls -l dump.d/databases/5/
total 2316
-rw-r--r--. 1 postgres postgres 2362652 Feb 27 14:46 4395.dat.gz
-rw-r--r--. 1 postgres postgres    25 Feb 27 14:46 4396.dat.gz
-rw-r--r--. 1 postgres postgres   2568 Feb 27 14:46 toc.dat
$
```

3.4.6. Pg_restore

The following options have been added to the pg_restore command.

□ Option --no-globals

The --no-globals option has been added to prevent the restoration of global objects (roles and tablespaces). This requires the pg_dumpall command output file (in a format other than text) as backup data. [[3c19983](#)]

Example 115 Specifying the --no-globals option

```
$ pg_restore -C --no-globals --format=d dump.d -d postgres -v
pg_restore: skipping restore of global objects because --no-globals was
specified
pg_restore: found database "template1" (OID: 1) in file "dump.d/map.dat"
...
pg_restore: executing SELECT pg_catalog.set_config('search_path', '', false);
pg_restore: creating TABLE "public.data1"
pg_restore: processing data for table "public.data1"
pg_restore: creating CONSTRAINT "public.data1_data1_pkey"
pg_restore: creating ACL "SCHEMA public"
pg_restore: number of restored databases is 2
```

□ Option --globals-only

The --globals-only option has been added to the pg_dumpall command to restore only global data. [[763aaa0](#)]

3.4.7. Pg_upgrade

The following features have been added to the pg_upgrade command.

□ Migration of In-Place Tablespaces

In-place tablespaces can now be migrated to the new version of the database cluster. In previous versions, in-place tablespaces were not copied. To use this feature, set the allow_in_place_tablespaces parameter to "on" in the destination database before migration. The presence of in-place tablespaces is not verified by the pg_upgrade --check command. [[3bcfd8](#)]

□ Speed Up Large Object Migration

The pg_upgrade command performs a bulk migration of data by executing the COPY FROM stdin statement when migrating the pg_largeobject_metadata and pg_shdepend catalogs. In previous

versions, it executed the `lo_create` function, `ALTER LARGE OBJECT` statements, and `GRANT` statements for each large object. [\[b33f753\]](#)

3.4.8. Pg_verifybackup

The `pg_verifybackup` command can now be executed on backups created in the tar format. In previous versions, the `--no-parse-wal` option was required to skip parsing WAL files. [\[b3cf461, f8a0cd2, b15c151\]](#)

Example 116 The `pg_verifybackup` command for tar-format backups

```
$ pg_basebackup --format=t --pgdata=back
$ pg_verifybackup --format=t --progress back
24984/24984 kB (100%) verified
backup successfully verified
```

3.4.9. Pg_waldump

It is now possible to run the `pg_waldump` command on backups created in the tar format. [\[b15c151\]](#)

Example 117 The `pg_waldump` command for tar-format backups

```
$ pg_basebackup --format=t -D back
$ pg_waldump back/pg_wal.tar --start=0/08000028 --end=0/08000088
rmgr: XLOG          len (rec/tot):   34/   34, tx:           0, lsn: 0/08000...
rmgr: Standby       len (rec/tot):   54/   54, tx:           0, lsn: 0/08000...
```

3.4.10. Psql

The following new features have been implemented in the `psql` command.

□ SERVICEFILE variable

The variable `SERVICEFILE`, which indicates the service file used for the connection, has been added. [\[6b1c4d3\]](#)

Example 118 Displaying the SERVICEFILE variable

```
$ psql "servicefile=/tmp/serviceA.conf service=service1"
psql (19beta1)
Type "help" for help.

postgres=# \echo :SERVICEFILE
/tmp/serviceA.conf
```

□ The %S substitution variable

It is now possible to specify the %S substitution variable in PROMPT1, PROMPT2, and PROMPT3, which define the input prompt for the psql command. This variable is substituted with the value of the search_path parameter. [\[b3ce55f\]](#)

Example 119 Changing PROMPT1 variables

```
postgres=> SHOW search_path ;
-[ RECORD 1 ]-----
search_path | "$user", public

postgres=> \set PROMPT1 ' [%S]=> '
["$user", public]=> SET search_path = 'public' ;
SET
[public]=>
```

□ The %i substitution variable

It is now possible to specify the substitution variable %i in PROMPT1, PROMPT2, and PROMPT3, which define the input prompts for the psql command. This variable is substituted based on the in_hot_standby parameter. [\[dddbbc2\]](#)

Table 36 Setting the %i substitution variable

In_hot_standby setting	%i placeholder	Note
on	primary	
off	standby	
-	?	Versions without in_hot_standby

Example 120 Changing PROMPT1 variables

```
postgres=> SHOW in_hot_standby ;
-[ RECORD 1 ]--+-----
in_hot_standby | off

postgres=> \set PROMPT1 ' [%i]=> '
[primary]=>
```

□ Outputting BOOLEAN Values

It is now possible to specify the output text for BOOLEAN values using the \pset command. Change the variables display_true and display_false. The default values for these variables are "f" (for false) and "t" (for true). [[645cb44](#)]

Example 121 Outputting BOOLEAN values

```
postgres=> \pset display_true 'True'
Boolean true display is "True".
postgres=> \pset display_false 'False'
Boolean false display is "False".
postgres=> SELECT true::boolean, false::boolean ;
  bool | bool
-----+-----
  True | False
(1 row)
```

□ Comment Output

Comments added to publications, extended statistics, and subscriptions are now included in the output of the \dRp+, \dRs+, and \dX+ metacommands. [[aecc558](#)]

Example 122 Comment output

```
postgres=> CREATE PUBLICATION pub1 FOR TABLE data1 ;
CREATE PUBLICATION
postgres=> COMMENT ON PUBLICATION pub1 IS 'comment for pub1' ;
COMMENT
postgres=> \dRp+
```

Publication pub1						
Owner	All tables	All sequences	Inserts	...	Via root	<u>Description</u>
demo	f	f	t	...	f	comment for pub1

Tables:

 "public.data1"

□ Publication Information

The table definition output now includes information indicating whether a table is included in a publication or not. [[2e1d4fd](#)]

Example 123 Comment output

```
postgres=# CREATE TABLE data1(col1 INT PRIMARY KEY, col2 VARCHAR(10)) ;
CREATE TABLE
postgres=# CREATE PUBLICATION pub1 FOR TABLE data1 ;
CREATE PUBLICATION
postgres=# CREATE PUBLICATION pub2 FOR ALL TABLES EXCEPT (TABLE data1) ;
CREATE PUBLICATION
postgres=# \d data1
```

Table "public.data1"				
Column	Type	Collation	Nullable	Default
col1	integer		not null	
col2	character varying(10)			

Indexes:

"data1_pkey" PRIMARY KEY, btree (col1)

Included in publications:

"pub1"

Excluded from publications:

"pub2"

3.4.11. Vacuumdb

The following options have been added to the vacuumdb command.

□ Option --dry-run

The --dry-run option has been added to display SQL statements without executing them. [[d107176](#)]

Example 124 Specifying the --dry-run option

```
$ vacuumdb -d postgres --dry-run --schema=public
vacuumdb: Executing in dry-run mode.
No commands will be sent to the server.
vacuumdb: vacuuming database "postgres"
VACUUM (SKIP_DATABASE_STATS) public.data1;
VACUUM (SKIP_DATABASE_STATS) public.data2;
VACUUM (SKIP_DATABASE_STATS) public.data3;
VACUUM (ONLY_DATABASE_STATS);
$
```

□ Change in the behavior of the --analyze-only option

When the --analyze-only option is specified with the vacuumdb command, statistics are now collected for partition tables as well. In previous versions, partition tables were excluded from statistics collection. [[6429e5b](#)]

3.5. Contrib Modules

This section describes new features related to the Contrib module.

3.5.1. Auto_explain

The following parameters have been added to the auto_explain module.

□ Parameter log_extension_options

The auto_explain.log_extension_options parameter has been added. This parameter specifies the string to be passed to the EXPLAIN statement in additional modules such as pg_overexplain.

Example 125 Specifying the log_extension_options option

```
postgres=# LOAD 'auto_explain' ;
LOAD
postgres=# LOAD 'pg_overexplain' ;
LOAD
postgres=# SET auto_explain.log_extension_options = 'DEBUG' ;
SET
```

□ Parameter log_io

The parameter auto_explain.log_io has been added. The default value is "off". If this parameter is set to "on", I/O statistics for executed SQL statements will be logged. This feature is effective only when the parameter auto_explain.log_analyze is set to "on". [[61c36a3](#)]

□ Parameter log_extension_options

This includes additional options from external modules such as the pg_overexplain module and the pg_plan_advice module in the log.

Example 126 Specifying the log_io option

```
postgres=# SET auto_explain.log_analyze = on ;
SET
postgres=# SET auto_explain.log_io = on ;
SET
postgres=# SELECT COUNT(*) FROM data1 ;
 count
-----
10000000
(1 row)

postgres=# \! tail -6 data/log/postgresql.log
LOG:  duration: 341.638 ms  plan:
      Query Text: SELECT COUNT(*) FROM data1;
      Aggregate  (cost=179055.00..179055.01 rows=1 width=8) (actual time=...
        -> Seq Scan on data1  (cost=0.00..154055.00 rows=10000000 width=...
          Prefetch: avg=46.37 max=91 capacity=94
          I/O: count=2397 waits=11 size=15.89 in-progress=4.27
```

3.5.2. Basic_archive

The system no longer checks for the existence of the directory specified by the `basic_archive.archive_directory` parameter at startup. This change will be backported to PostgreSQL 15 and later. [[cc774c5](#)]

3.5.3. File_fdw

The "header" option, which specifies the number of header records in the data file, has been added. [[26cb14a](#)]

Example 127 Reading the CSV file with a two-line header

```
postgres=# \! cat /tmp/multi1.csv
HEADER1-1,HEADER1-2
HEADER2-1,HEADER2-2
100,data1
200,data2
postgres=# CREATE EXTENSION file_fdw ;
CREATE EXTENSION
postgres=# CREATE SERVER filesvr1 FOREIGN DATA WRAPPER file_fdw ;
CREATE SERVER
postgres=# CREATE FOREIGN TABLE multi1(col1 INT, col2 VARCHAR(10)) SERVER
    filesvr1 OPTIONS (filename '/tmp/multi1.csv', HEADER '2', format 'csv') ;
CREATE FOREIGN TABLE
postgres=# SELECT * FROM multi1 ;
 col1 | col2
-----+-----
  100 | data1
  200 | data2
(2 rows)
```

3.5.4. Injection_points

The following new features have been added to the injection_points module.

□ Function injection_points_attach

The injection_points_attach function has been updated to include additional arguments. [[16a2f70](#)]

□ Function injection_points_list

The injection_points_list function, which returns a list of injection points, has been added. [[4eca711](#)]

□ Removed Functions

The following functions related to statistics information have been removed. [[d52c24b](#)]

- injection_points_stats_numcalls
- injection_points_stats_drop
- injection_points_stats_fixed

3.5.5. Pg_buffercache

The following new features have been implemented in the pg_buffercache module.

□ Add Views

The following views have been added. [[4b203d4](#)]

Table 37 Added Views

View name	Description
pg_buffercache_os_pages	Mapping between the buffer pool and OS pages
pg_buffercache_numa	Mapping of the buffer pool and NUMA nodes

□ Add Functions

The following functions have been added. [[9ccc049](#), [4b203d4](#)]

Table 38 Added Functions

Function name	Description
pg_buffercache_os_pages	Provides a mapping between the buffer pool and OS pages
pg_buffercache_mark_dirty	Sets the specified buffer to the dirty state
pg_buffercache_mark_dirty_relation	Sets the buffers for the specified relation in the buffer pool to the dirty state
pg_buffercache_mark_dirty_all	Sets all buffers in the buffer pool to the dirty state

Example 128 Executing of the pg_buffercache_mark_dirty_all function

```
postgres=# SELECT * FROM pg_buffercache_mark_dirty_all() ;
-[ RECORD 1 ]-----+-----
buffers_dirtied      | 5527
buffers_already_dirty | 10857
buffers_skipped       | 0
```

3.5.6. Pg_plan_advice

The new Contrib module `pg_plan_advice` has been added to PostgreSQL 19. This module provides features for changing execution plans and verifying the modified execution plans. [[5883ff3](#), [6455e55](#), [26255a3](#)]

□ Basic Usage

To use this module, follow the steps below.

1. Specify `pg_plan_advice` in the `LOAD` statement or in the `shared_preload_libraries` parameter.
2. Verify the execution plan using an `EXPLAIN` statement with the `PLAN_ADVICE` option.
3. Information for changing the execution plan is output in the "Generated Plan Advice:" section.
4. Specify information regarding changes to the execution plan in the `pg_plan_advice.advice` parameter.
5. The execution plan will be changed.

In the following example, the join order of tables `data1` and `data2` is changed to verify that the execution plan changes.

Example 129 Verifying the Method for Modifying the Execution Plan

```
postgres=# LOAD 'pg_plan_advice' ;
LOAD
postgres=# EXPLAIN (COSTS OFF, PLAN_ADVICE) SELECT * FROM data1 AS d1
           INNER JOIN data2 AS d2 ON d1.col1 = d2.col1 WHERE d1.col1 < 1000 ;
           QUERY PLAN
```

Nested Loop

```
-> Index Scan using data1_pkey on data1 d1
    Index Cond: (col1 < 1000)
-> Index Scan using data2_pkey on data2 d2
    Index Cond: (col1 = d1.col1)
```

Generated Plan Advice:

```
JOIN_ORDER(d1 d2)
NESTED_LOOP_PLAIN(d2)
INDEX_SCAN(d1 public.data1_pkey d2 public.data2_pkey)
NO_GATHER(d1 d2)
(10 rows)
```

Example 130 Configure Execution Plan Changes

```
postgres=# SET pg_plan_advice.advice = 'JOIN_ORDER(d2 d1)' ;
SET
postgres=# EXPLAIN (COSTS OFF) SELECT * FROM data1 AS d1 INNER JOIN data2 AS
           d2 ON d1.col1 = d2.col1 WHERE d1.col1 < 1000 ;
           QUERY PLAN

-----

Gather
  Workers Planned: 2
    -> Hash Join
          Hash Cond: (d2.col1 = d1.col1)
            -> Parallel Seq Scan on data2 d2
            -> Hash
                  -> Index Scan using data1_pkey on data1 d1
                      Index Cond: (col1 < 1000)

Supplied Plan Advice:
  JOIN_ORDER(d2 d1) /* matched */
(10 rows)
```

Specify the changes to the execution plan in the `pg_plan_advice.advice` parameter. The major settings are listed below.

Table 39 Typical "advice" parameter settings

Setting value	Description
SEQ_SCAN(target)	Use the sequential scan (Seq Scan)
INDEX_SCAN(target)	Use the specified index
JOIN_ORDER(item item...)	Specify the join order
HASH_JOIN(target)	Execute the hash join
NESTED_LOOP_PLAN(target)	Execute the nested loop join
GATHER(item)	Specify the parallel query
NO_GATHER(item)	Do not use a parallel query
DO_NOT_SCAN(item)	Stop scanning the specified relation

□ Parameters

The `pg_plan_advice` module provides the following custom parameters.

Table 40 追加されたパラメーター

Parameter name	Description	Default value
<code>advice</code>	Advice on changing the execution plan	"
<code>always_explain_supplied_advice</code>	Always include output indicating which child has had its execution plan changed in the results of the EXPLAIN statement	on
<code>always_store_advice_details</code>	Whether to output feedback for prepared statements	off
<code>feedback_warnings</code>	Whether to issue a warning if the advice was not successfully applied	off
<code>trace_mask</code>	Whether to output debug traces	off

If the `always_explain_supplied_advice` parameter is set to "on" (the default value), the EXPLAIN statement will always output information about any changes to the execution plan (under the heading "Supplied Plan Advice:"). If this parameter is set to "off", specifying the `PLAN_ADVICE` option in the EXPLAIN statement is required to output information about changes to the execution plan.

3.5.7. Pg_stash_advice

The new Contrib module `pg_stash_advice` has been added to PostgreSQL 19. The `pg_stash_advice` module applies execution plan advice to query IDs. By default, the generated advice is stored in memory and is not persisted. To persist the advice stash data, set the `pg_stash_advice.persist` parameter to "true". The persisted data is created as the file `pg_stash_advice.tsv` in the database cluster. [[e8ec19a](#), [c10edbl](#)]

Table 41 Added Functions

Function name	Description	Note
<code>pg_create_advice_stash</code>	Creating the Advice Stash	
<code>pg_drop_advice_stash</code>	Dropping the Advice Stash	
<code>pg_set_stashed_advice</code>	Setting advice for a query	
<code>pg_get_advice_stashes</code>	List of Advice Stashes	
<code>pg_get_advice_stash_contents</code>	Viewing the contents of the Advice Stash	

The following parameters are defined in this module.

Table 42 Added Parameters

Parameter name	Description	Default value
pg_stash_advice.persist	Whether to persist advice	false
pg_stash_advice.stash_name	Name of the active advice stash	"
pg_stash_advice.persist_interval	Persistence check interval	30s

□ Preparation

Set the `shared_preload_libraries` parameter to `pg_stash_advice`. Set the `compute_query_id` parameter to "auto" (default value) or "on". Next, enable the module using the `CREATE EXTENSION` statement.

Example 131 Preparing pg_stash_advice module

```
postgres=# SHOW shared_preload_libraries ;
shared_preload_libraries
-----
pg_stash_advice
(1 row)

postgres=# SHOW compute_query_id ;
compute_query_id
-----
on
(1 row)

postgres=# CREATE EXTENSION pg_stash_advice ;
CREATE EXTENSION
```

Create the advice stash. Specify the name for the `pg_create_advice_stash` function, and set the name of the advice just created to the `pg_advice_stash.stash_name` parameter.

Example 132 Creating and Configuring a Advice Stash

```
postgres=# SELECT pg_create_advice_stash('stash1') ;
pg_create_advice_stash
-----

(1 row)

postgres=# SET pg_stash_advice.stash_name = 'stash1' ;
SET
```

Identify the query ID and execution plan of the query for which the execution plan needs to be changed.

Example 133 Checking the Query ID

```
postgres=# EXPLAIN (VERBOSE, COSTS OFF) SELECT * FROM data1 AS d1 INNER
JOIN data2 AS d2 ON d1.col1 = d2.col1 WHERE d1.col1 = 10000 ;
QUERY PLAN
-----

Nested Loop
Output: d1.col1, d1.col2, d2.col1, d2.col2
-> Index Scan using data1_pkey on public.data1 d1
    Output: d1.col1, d1.col2
    Index Cond: (d1.col1 = 10000)
-> Index Scan using data2_pkey on public.data2 d2
    Output: d2.col1, d2.col2
    Index Cond: (d2.col1 = 10000)
Query Identifier: 1524396781679940627
(9 rows)
```

Specify the advice to change the execution plan for the query ID. In the following example, the join order is changed. For instructions on changing the execution plan, refer to the `pg_plan_advice` Contrib module.

Example 134 Specify advice

```
postgres=# SELECT pg_set_stashed_advice('stash1', 1524396781679940627,  
      'JOIN_ORDER(d2 d1)') ;  
pg_set_stashed_advice  
-----  
  
(1 row)
```

Check the execution plan. The "Supplied Plan Advice" entry is output, indicating that advice for changing the execution plan has been specified.

Example 135 Check the Execution Plan

```
postgres=# EXPLAIN (VERBOSE, COSTS OFF) SELECT * FROM data1 AS d1 INNER JOIN  
data2 AS d2 ON d1.col1 = d2.col1 WHERE d1.col1 = 10000 ;  
QUERY PLAN  
-----  
Nested Loop  
  Output: d1.col1, d1.col2, d2.col1, d2.col2  
  -> Index Scan using data2_pkey on public.data2 d2  
      Output: d2.col1, d2.col2  
      Index Cond: (d2.col1 = 10000)  
  -> Index Scan using data1_pkey on public.data1 d1  
      Output: d1.col1, d1.col2  
      Index Cond: (d1.col1 = 10000)  
Query Identifier: 1524396781679940627  
Supplied Plan Advice:  
  JOIN_ORDER(d2 d1) /* matched */  
(11 rows)
```

3.5.8. Pg_stat_statements

The following new features have been added to the pg_stat_statements module.

- Changes in the pg_stat_statements view

The following columns have been added to the pg_stat_statements view. [[3357471](#)]

Table 43 Added Columns

Column name	Data type	Description
generic_plan_calls	bigint	Number of executions of the generic execution plan
custom_plan_calls	bigint	Number of executions of the custom execution plan

Example 136 Number of Execution Plans Executed

```
postgres=> SELECT query, generic_plan_calls, custom_plan_calls FROM
              pg_stat_statements WHERE query LIKE 'PREPARE%' ;
-[ RECORD 1 ]-----+-----
query              | PREPARE prepare1(int) AS SELECT $1
generic_plan_calls | 4
custom_plan_calls  | 4
```

- Constancy of FETCH Statements

The number of rows specified in the FETCH statement is now treated as the constant and assigned the single query ID. [[bee23ea](#)]

Example 137 Constancy of FETCH Statements

```
postgres=> BEGIN ;
BEGIN
postgres=> DECLARE c1 CURSOR FOR SELECT * FROM data1 ;
DECLARE CURSOR
postgres=> FETCH 1 FROM c1 ;
  c1 |  c2
-----+-----
   1 | data1
(1 row)
postgres=> FETCH 2 FROM c1 ;
  c1 |  c2
-----+-----
   2 | data2
   3 | data3
(2 rows)
postgres=> SELECT query, calls FROM pg_stat_statements WHERE query LIKE
'FETCH%' ;
      query      | calls
-----+-----
  FETCH $1 FROM c1 |      2
(1 row)
```

3.5.9. Postgres_fdw

The following features have been added to the postgres_fdw module.

□ Option restore_stats

The Foreign Data Wrapper now supports importing remote statistics via the callback. The postgres_fdw module implements this callback, and the restore_stats option can be specified for both the FOREIGN SERVER and the FOREIGN TABLE. [[28972b6](#)]

Example 138 Specifying the restore_stats option

```
postgres=> CREATE FOREIGN TABLE data1(col1 INT, col2 VARCHAR(10)) SERVER dbsvr1
           OPTIONS (restore_stats 'true') ;
CREATE FOREIGN TABLE
postgres=> ANALYZE (VERBOSE) data1 ;
INFO:  importing statistics for foreign table "public.data1"
INFO:  finished importing statistics for foreign table "public.data1"
ANALYZE
```

□ Pushdown of ARRAY Operations

When using a generic plan, the ANY clause in the PREPARE statement can now be pushed down.

[\[62c3b4c\]](#)

Example 139 SQL statement to execute

```
postgres=> SET plan_cache_mode = force_generic_plan ;
SET
postgres=> PREPARE array_stmt(VARCHAR[]) AS SELECT * FROM data1 WHERE
           col2 = ANY($1) ;
PREPARE
postgres=> EXPLAIN (VERBOSE, COSTS OFF) EXECUTE
           array_stmt(ARRAY['data1', 'data2']) ;
```

Example 140 PostgreSQL 18 Execution Plan

```
postgres=> EXPLAIN (VERBOSE, COSTS OFF) EXECUTE
           array_stmt(ARRAY['data1', 'data2']) ;
           QUERY PLAN

-----
Foreign Scan on public.data1
  Output: col1, col2
  Filter: ((data1.col2)::text = ANY (($1)::text[]))
  Remote SQL: SELECT col1, col2 FROM public.data1
(4 rows)
```

Example 141 PostgreSQL 19 Execution Plan

```
postgres=> EXPLAIN (VERBOSE, COSTS OFF) EXECUTE
           array_stmt(ARRAY['data1', 'data2']) ;
           QUERY PLAN

-----
Foreign Scan on public.data1
  Output: col1, col2
  Remote SQL: SELECT col1, col2 FROM public.data1 WHERE ((col2 = ANY
($1::character_varying[])))
(3 rows)
```

□ Transaction State Propagation

The status of local READ ONLY and DEFERRABLE transactions is now pushed down to remote connections. [[de28140](#)]

3.5.10. Miscellaneous

In addition to what was described above, the following test extension modules have been added to the src/test/modules directory. [[31280d9](#), [e0e4c13](#)]

Table 44 Added extension modules

Module name	Description	Note
test_binaryheap	Binary heap testing	
test_bitmapset	Bitmapset testing	
test_checksums	Checksum behavior testing	
test_cplusplusxt	C++ header compilation testing	
test_custom_stats	Fixed- and variable-size custom statistics testing	
test_custom_types	Custom type testing	
test_int128	128-bit int type testing	
test_lwlock_tranches	LWLock Tranche testing	
test_plan_advice	pg_plan_advice module testing	
test_saslprep	SASLprep implementation testing	
test_shmem	Shared memory testing	

URL list

The following websites are references to create this material.

- PostgreSQL 19 Release Notes
<https://www.postgresql.org/docs/19/release.html>
- PostgreSQL 19 Manual
<https://www.postgresql.org/docs/19/index.html>
- PostgreSQL 19 Beta 1 Announcement
<https://www.postgresql.org/about/news/postgresql-19-beta-1-released-3313/>
- PostgreSQL 19 Open Items
https://wiki.postgresql.org/wiki/PostgreSQL_19_Open_Items
- pgPedia
<https://pgpedia.info/postgresql-versions/postgresql-19.html>
- Commitfests
<https://commitfest.postgresql.org/>
- Git
`git://git.postgresql.org/git/postgresql.git`
- GitHub
<https://github.com/postgres/postgres>
- pgsql-hackers Mailing list
<https://www.postgresql.org/list/pgsql-hackers/>
- PostgreSQL Developer Information
https://wiki.postgresql.org/wiki/Development_information
- Slack - postgresql-jp (Japanese)
<https://postgresql-jp.slack.com/>

Change History

Change history

Version	Date	Author	Description
1.0	June 4, 2026	Noriyoshi Shinoda	Created for PostgreSQL 19 Beta 1
1.0.1	Jun 6, 2026	Noriyoshi Shinoda	Drop 3.2.1. ALTER DEFAULT PRIVILEGE statement

